Deploy de infraestructure

Configuración de vault con k8s y Jenkins

Primero validamos que exista el certificado

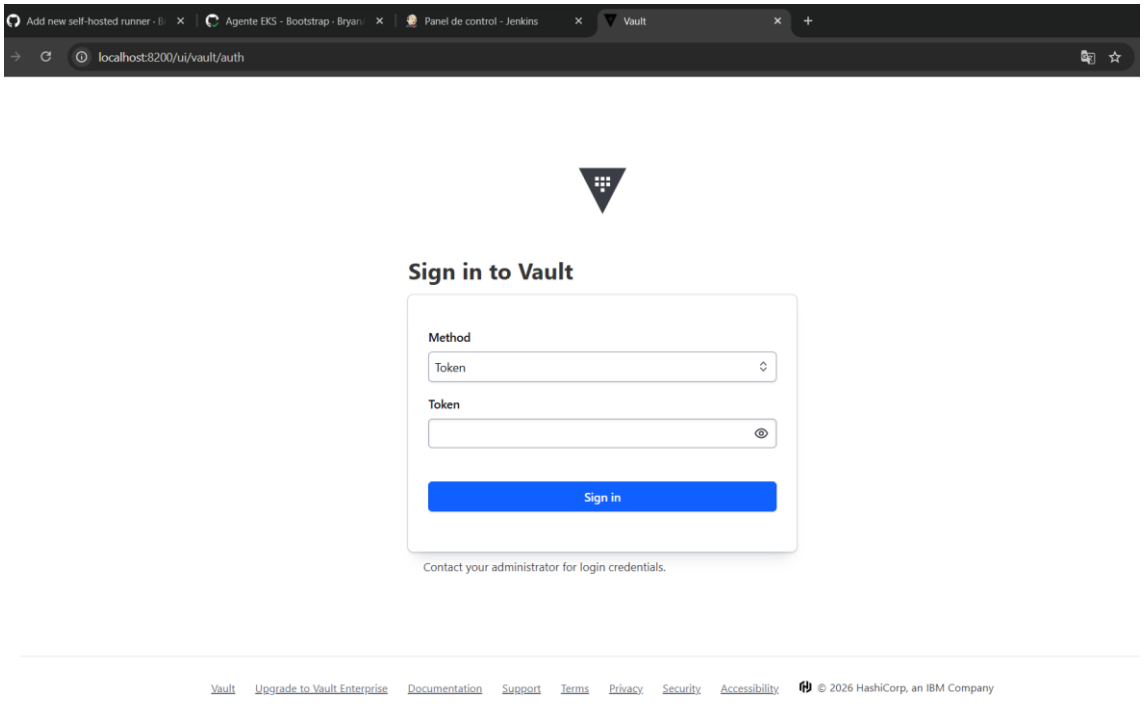
```
❯ kubectl exec -n prueba-cusca vault-0 -- \
  sh -c 'ls -l /var/run/secrets/kubernetes.io/serviceaccount/'
total 0
lrwxrwxrwx    1 root     vault          13 Oct 10 23:15 ca.crt -> ..data/ca.crt
lrwxrwxrwx    1 root     vault          16 Oct 10 23:15 namespace -> ..data/namespace
lrwxrwxrwx    1 vault    vault          12 Oct 10 23:15 token -> ..data/token

/mnt/c/Users/Bryan/Desktop/workspace/cusca/AW1020001_AgentSetting_DevOps
❯
```

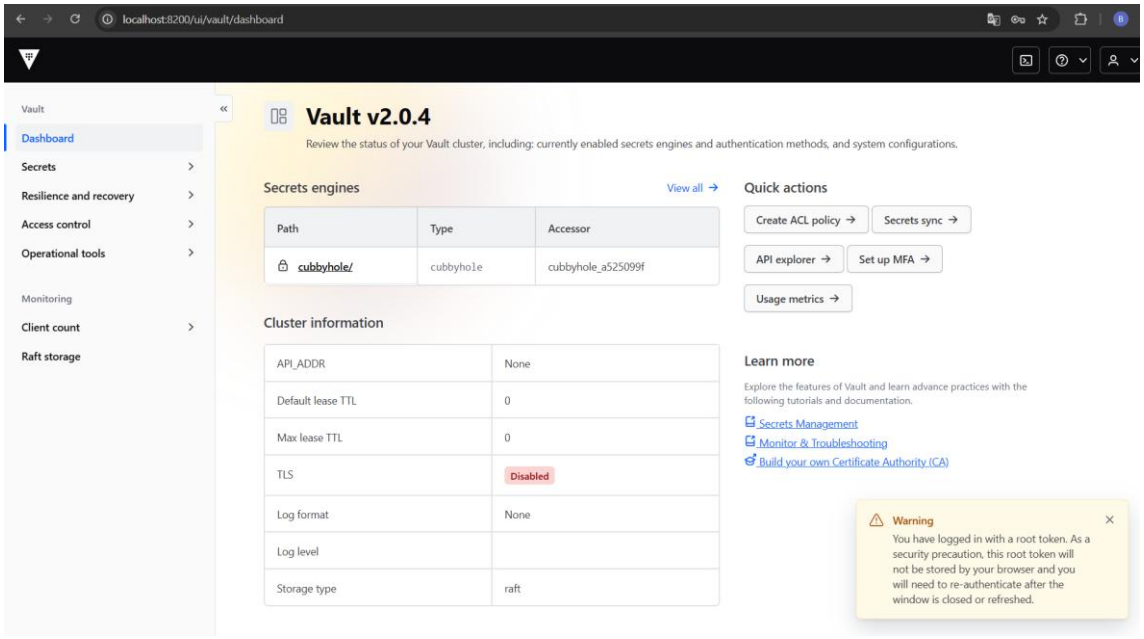
Luego nos conectamos al vault, de mi parte hice un portforward para acceder a este ya que corre en localhost dentro del cluster

```
❯ kubectl port-forward -n prueba-cusca svc/vault 8200:8200
Forwarding from 127.0.0.1:8200 -> 8200
Forwarding from [::1]:8200 -> 8200
Handling connection for 8200
Handling connection for 8200
Handling connection for 8200
Handling connection for 8200
Handling connection for 8200
Handling connection for 8200
Handling connection for 8200
Handling connection for 8200
Handling connection for 8200
```

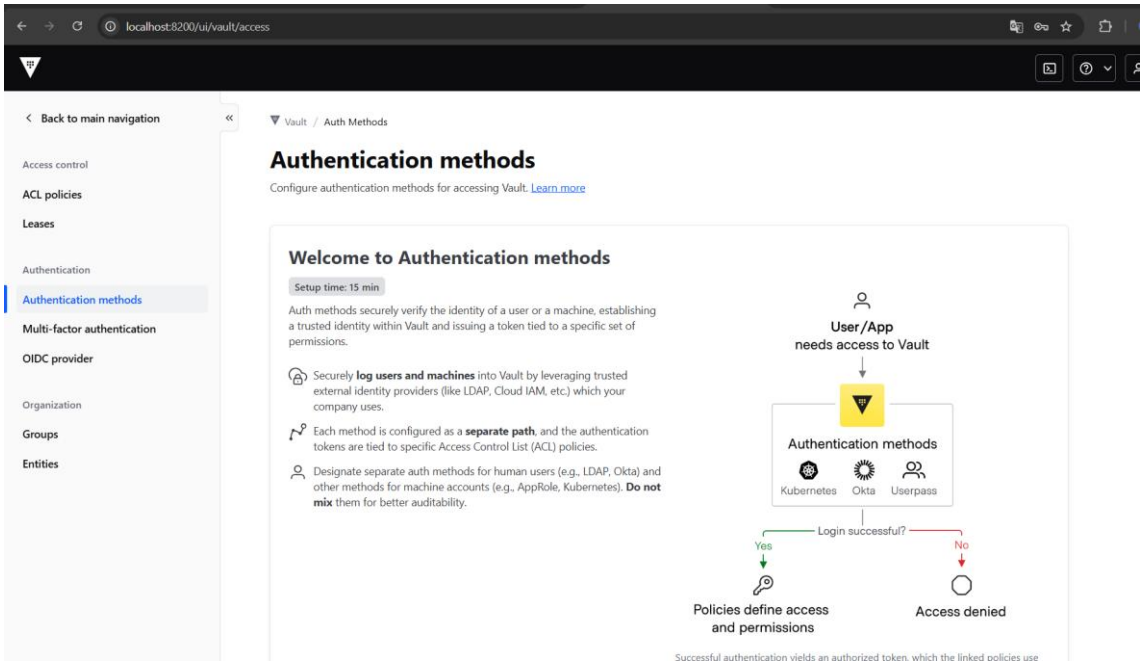
Luego de eso, accedí por la ruta http://127.0.0.1:8200,

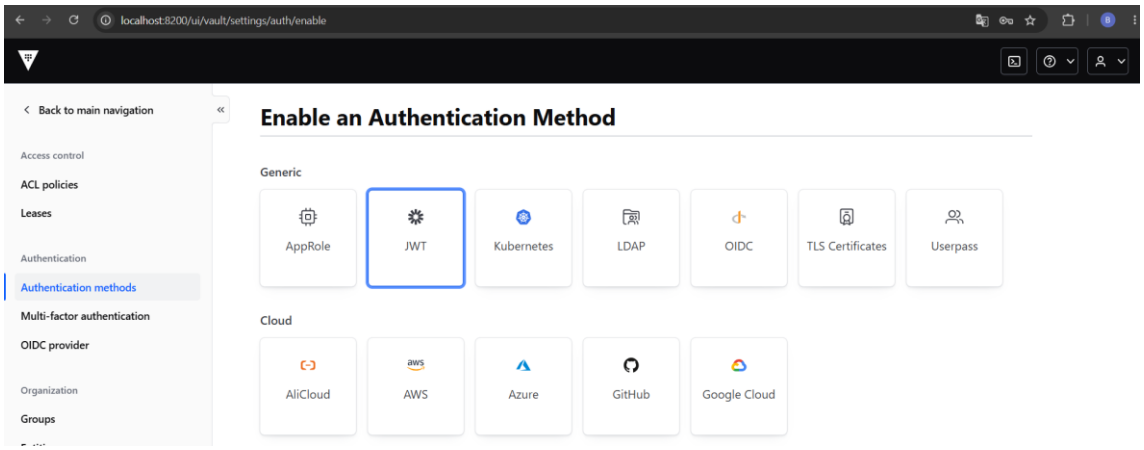


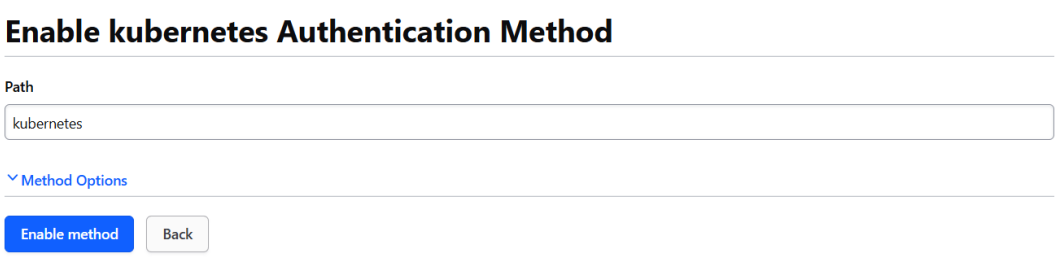
Luego accedí por medio de token, una vez dentro en limpio recién creado, de momento se tiene configurado sin TLS



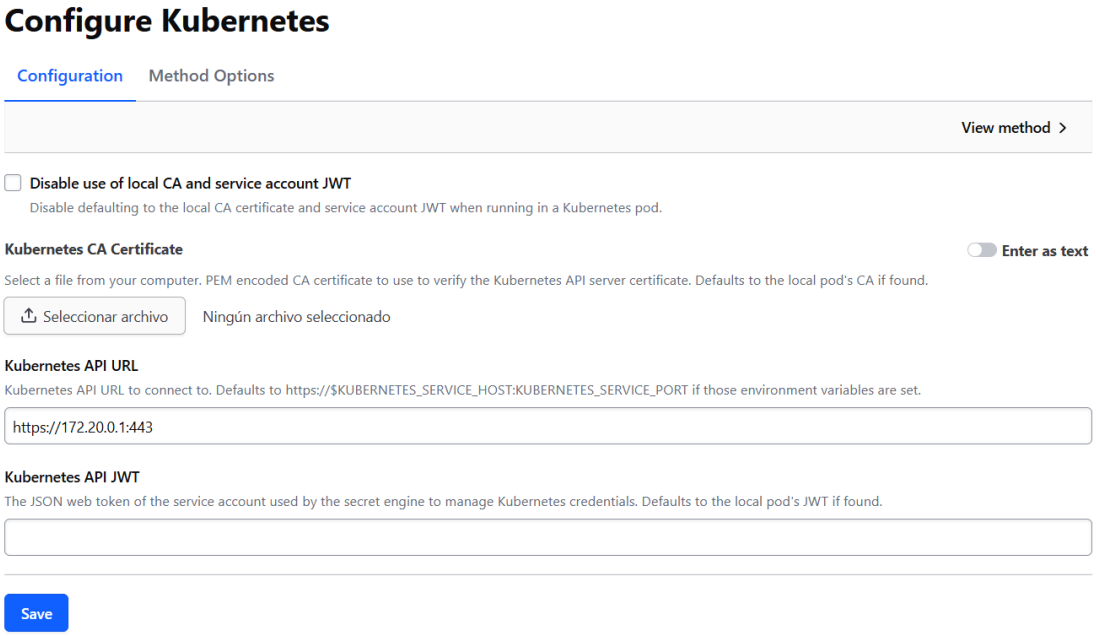
Para configurar el control de acceso en el apartado Access control > authentication method, agregue un metodo por medio de kubernetes

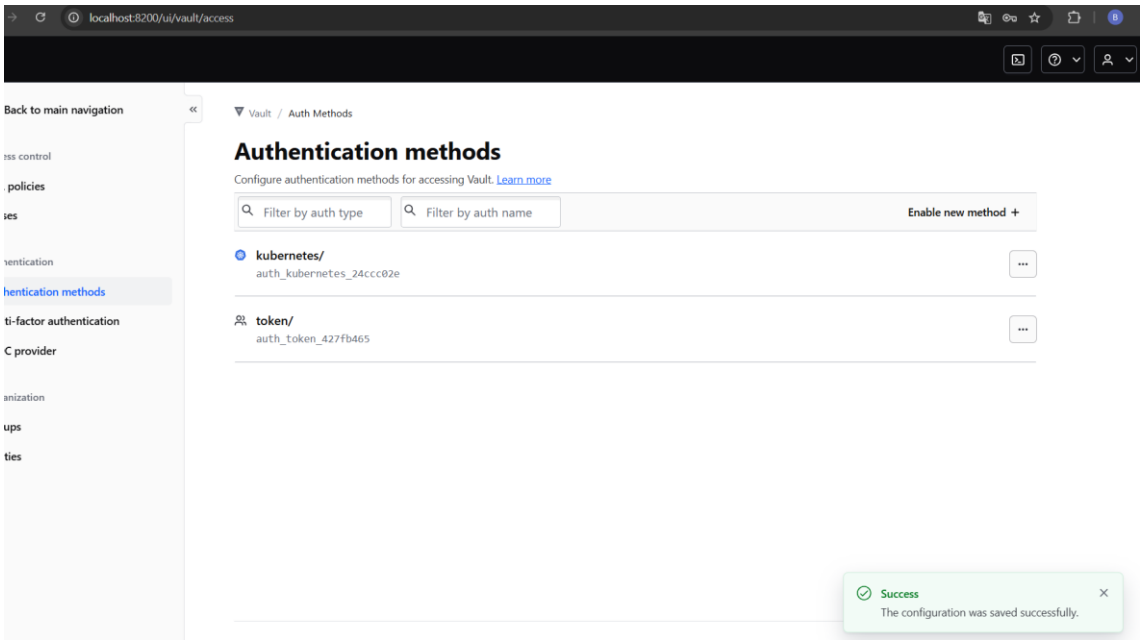




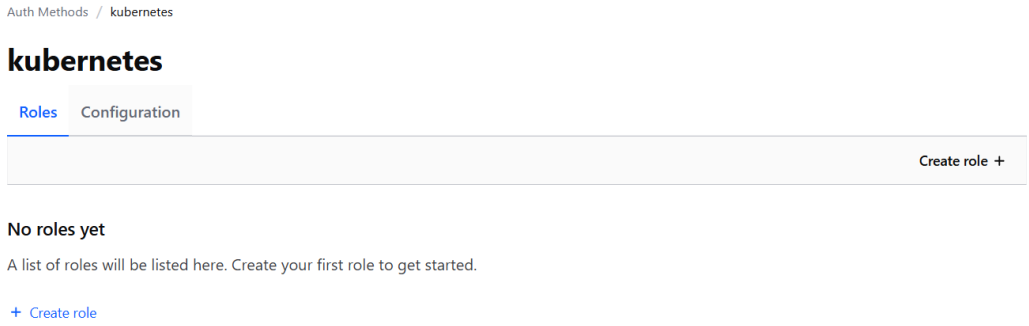


Y se agregó la siguiente configuración





Luego de esto se procede a crear un role para la auth de Jenkins accediendo al metodo kubernetes



Se agrega la siguiente configuración

Name

Name of the role.

jenkins

Alias name source

Source to use when deriving the Alias name. valid choices: "serviceaccount_uid" : <token.uid> e.g. 474b11b5-0f20-4f9d-8ca5-65715ab325e0 (most secure choice) "serviceaccount_name" : <namespace>/<serviceaccount> e.g. vault/vault-agent default: "serviceaccount_uid"

serviceaccount_uid

Audience

Optional Audience claim to verify in the jwt.

Bound service account names ⓘ

Add one item per row.

jenkins

Add

Bound service account namespace selector

A label selector for Kubernetes namespaces which are allowed to access this role. Accepts either a JSON or YAML object. If set with bound_service_account_namespaces, the conditions are ORed.

Bound service account namespaces ⓘ

Add one item per row.

prueba-cusca



Hide Tokens

Token Alias Metadata

The metadata to be tied to generated entity alias. This should be a list or map containing the metadata in key value pairs

```
1 {
2 }
```

Generated Token's Bound CIDRs ⓘ

Add one item per row.

Add

Generated Token's Explicit Maximum TTL

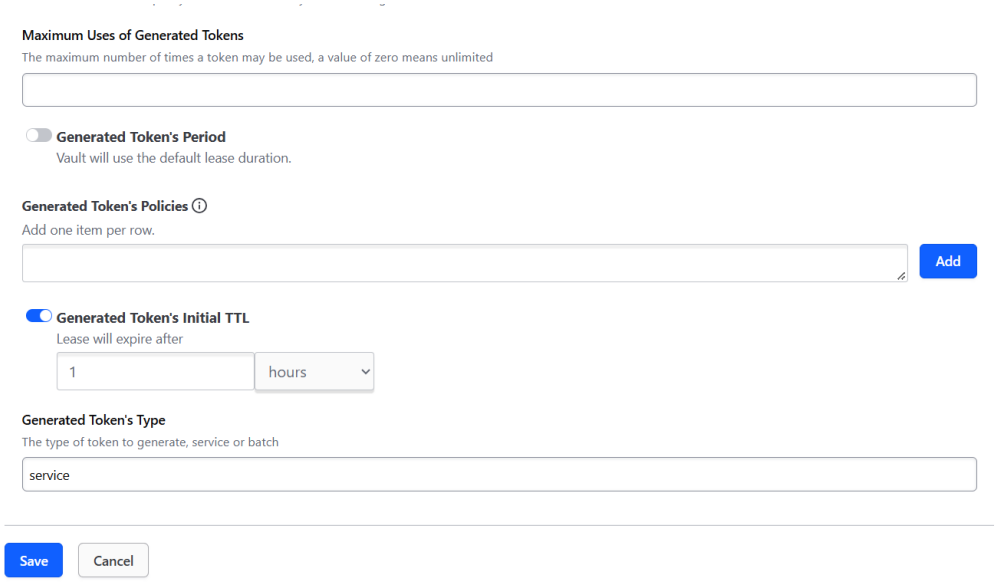
Vault will use the default lease duration.

Generated Token's Maximum TTL

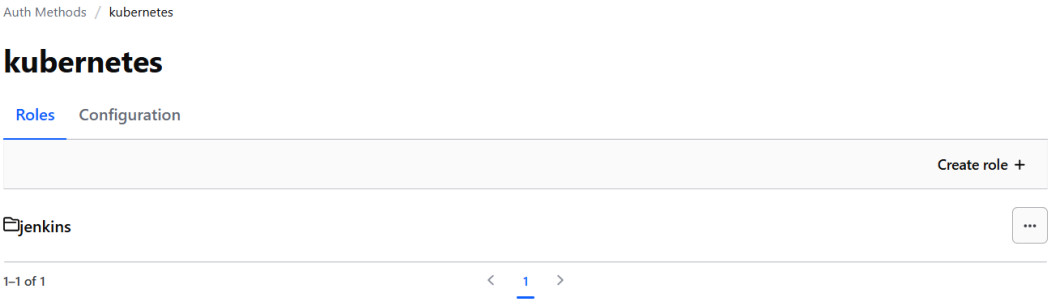
Vault will use the default lease duration.

Do Not Attach 'default' Policy To Generated Tokens

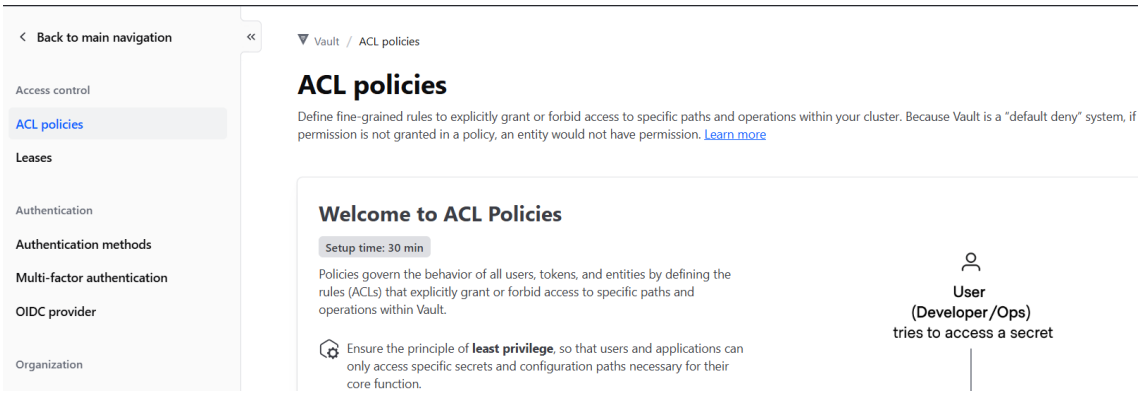
If true, the 'default' policy will not automatically be added to generated tokens

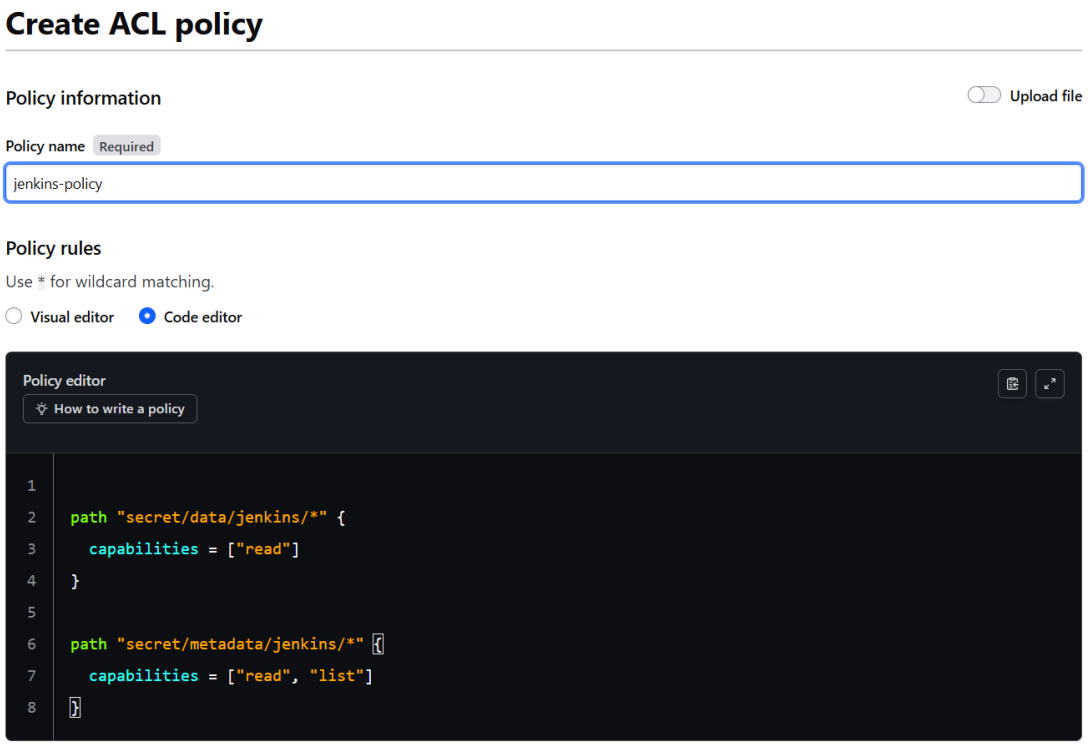


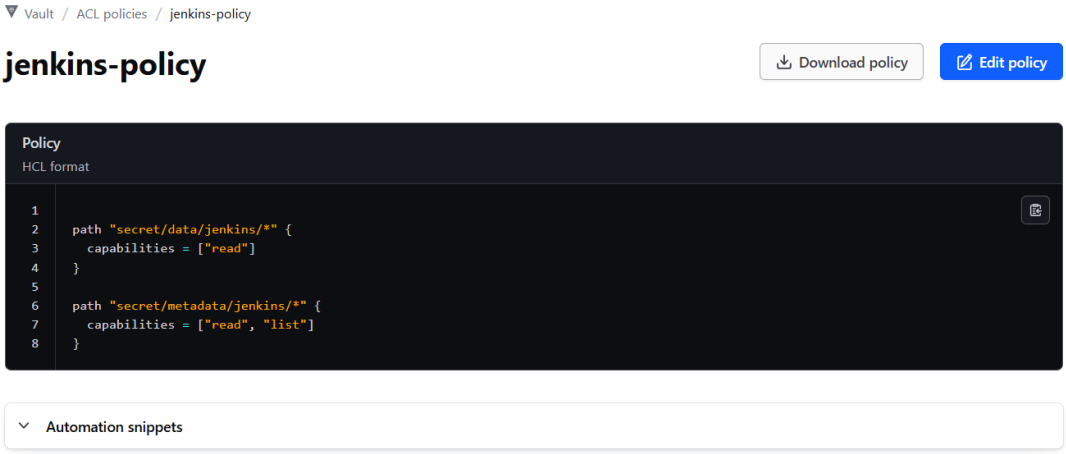
Una vez creemos el role se vera acá



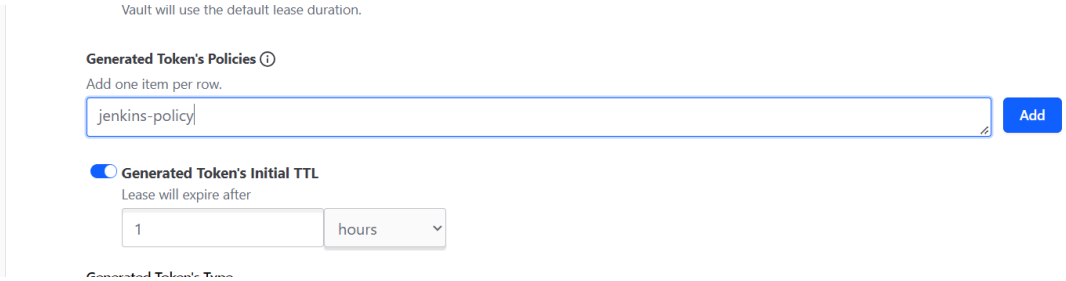
Se procede a crear una ACL policie



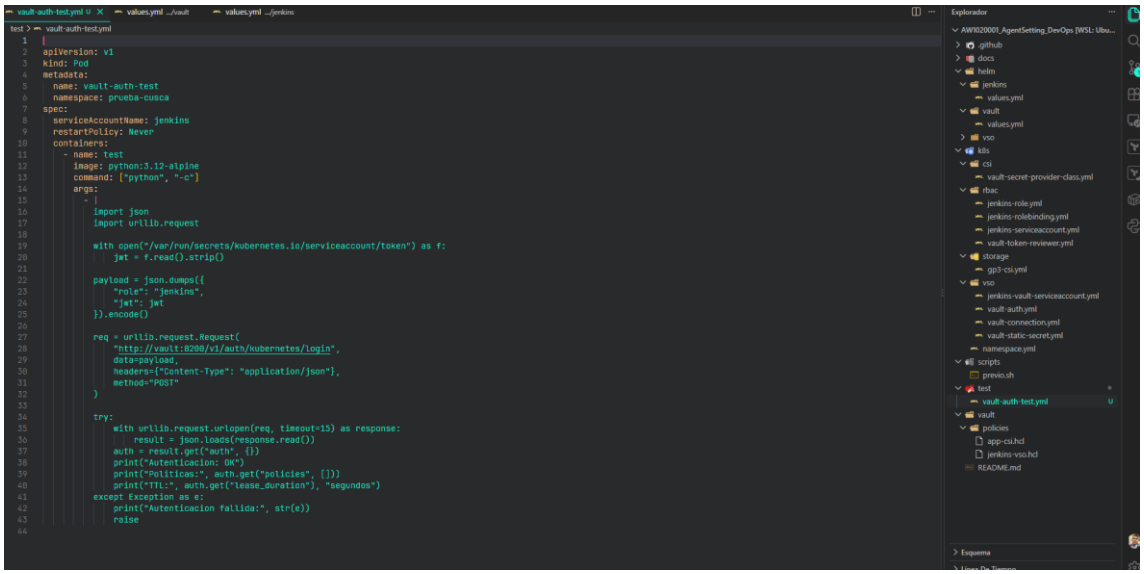




Por último, agregamos esta policy al role creado de Jenkins dentro del metodo de auth kubernetes



Luego de esto procedemos a agregar un nuevo manifiesto de prueba para validar la conexión



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: vault-auth-test
  namespace: prueba-cusca
spec:
  serviceAccountName: jenkins
  restartPolicy: Never
  containers:
    - name: test
      image: python:3.12-alpine
      command: ["python", "-c"]
      args:
        - |
          import json
          import urllib.request

          with open("/var/run/secrets/kubernetes.io/serviceaccount/token") as f:
              jwt = f.read().strip()

          payload = json.dumps({
              "role": "jenkins",
              "jwt": jwt
          }).encode()

          req = urllib.request.Request(
              "http://vault:8200/v1/auth/kubernetes/login",
              data=payload,
              headers={"Content-Type": "application/json"},
              method="POST"
          )

          try:
              with urllib.request.urlopen(req, timeout=15) as response:
                  result = json.loads(response.read())
              auth = result.get("auth", {})
              print("Autenticacion: OK")
              print("Politicas:", auth.get("policies", []))
              print("TTL:", auth.get("lease_duration"), "segundos")
          except Exception as e:
              print("Autenticacion fallida:", str(e))
              raise
```

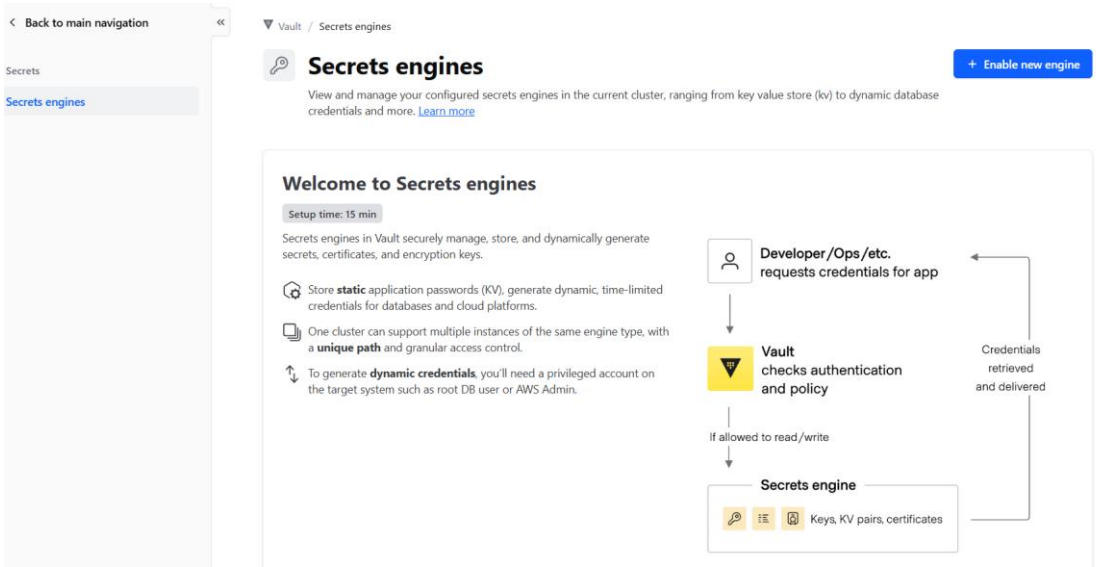
Hacemos un apply de este manifiesto y vemos su resultado donde nos dirá que tuvimos auth exitosa y que política tenemos, de esto tenemos todo listo



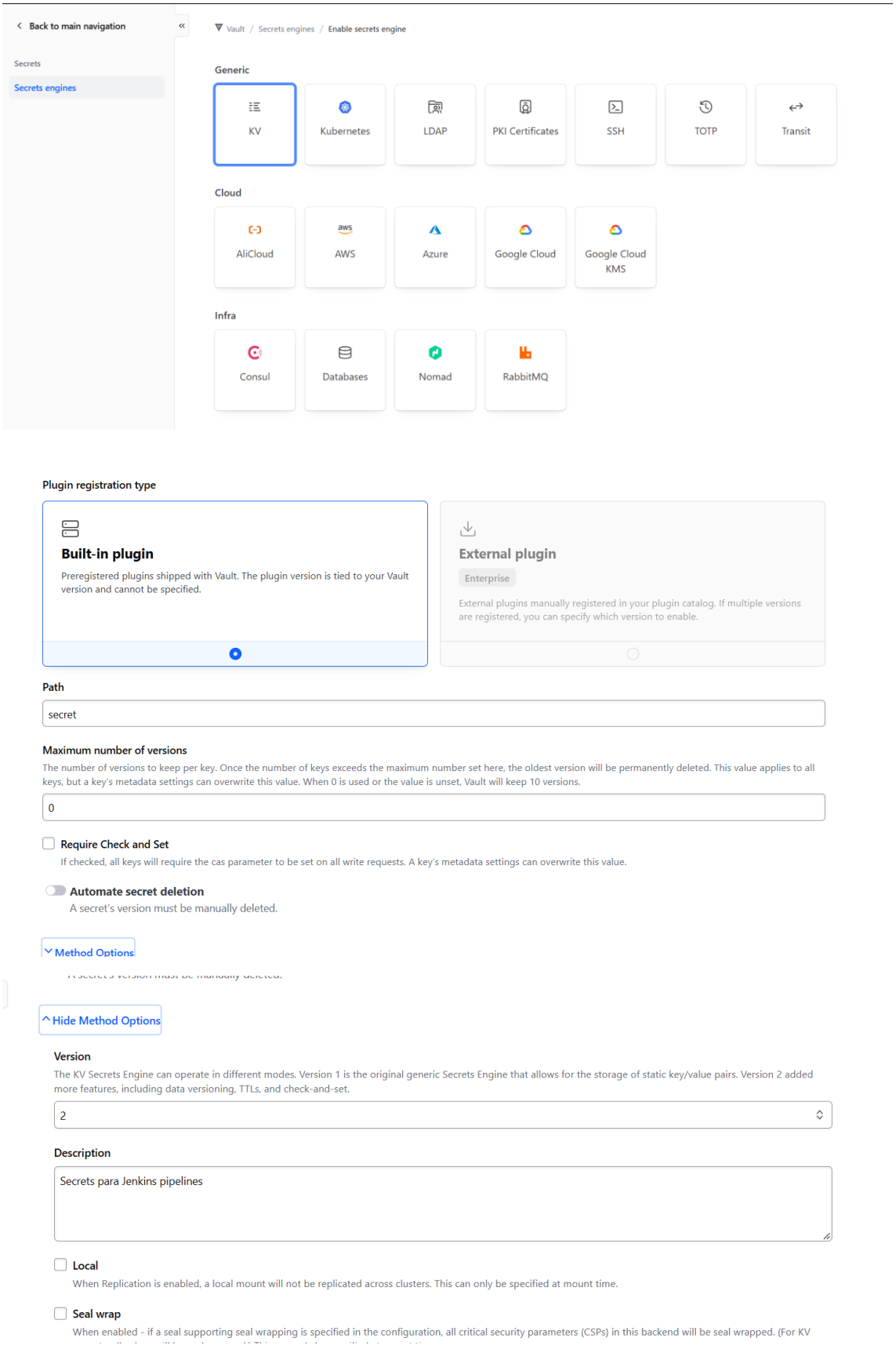
```
❯ kubectl apply -f vault-auth-test.yml
pod/vault-auth-test created
❯ kubectl wait -n prueba-cusca \
  --for=jsonpath='{.status.phase}'=Succeeded \
  pod/vault-auth-test --timeout=90s
pod/vault-auth-test condition met
❯ kubectl logs -n prueba-cusca vault-auth-test
Autenticacion: OK
Politicas: ['default', 'jenkins-policy']
TTL: 3600 segundos

/mnt/c/Users/Bryan/Desktop/workspace/cusca/AW1020001_AgentSetting_DevOps/test
❯
```

Luego de esto procedemos al vault a habilitar el secret engine para la tecnología necesaria

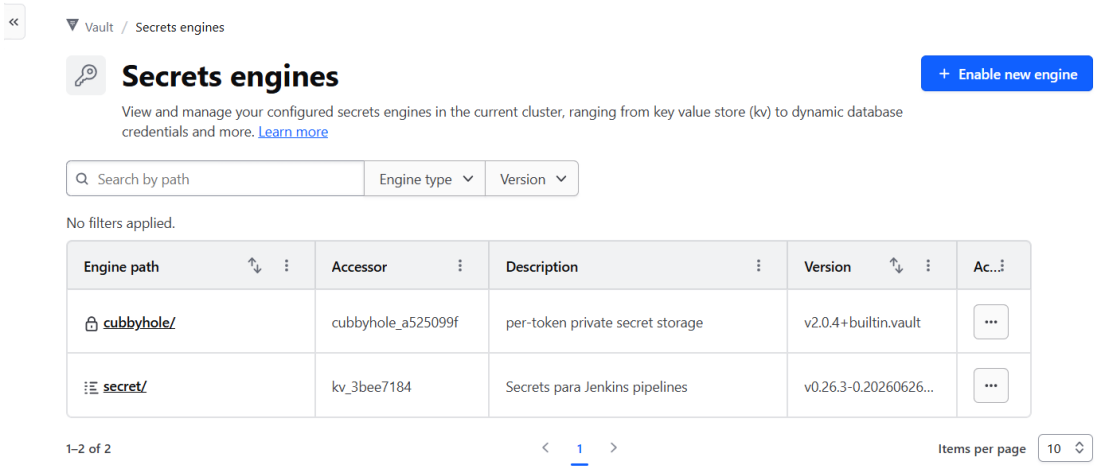


Seleccionaremos la opción KV (key-Value) hay que asegurarnos sea versión dos

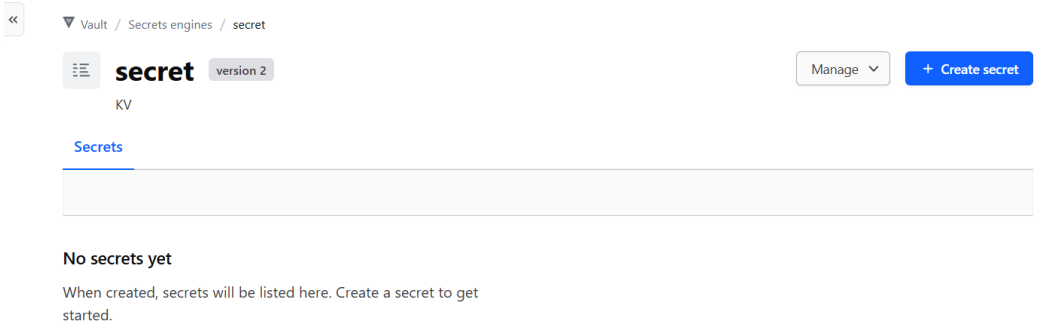


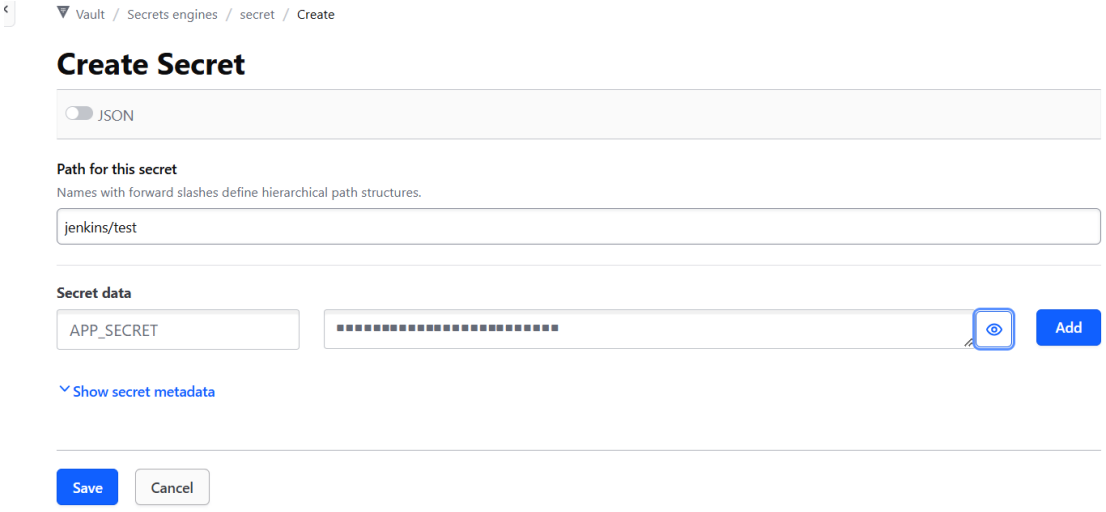
Las demás opciones dejarlas tal cual están

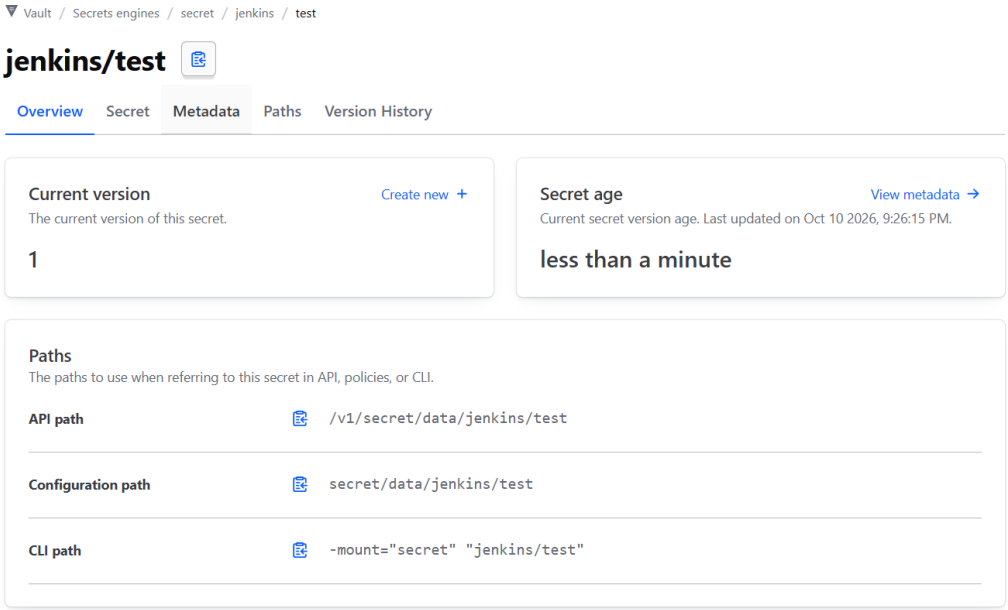
Validamos que se creara secret/

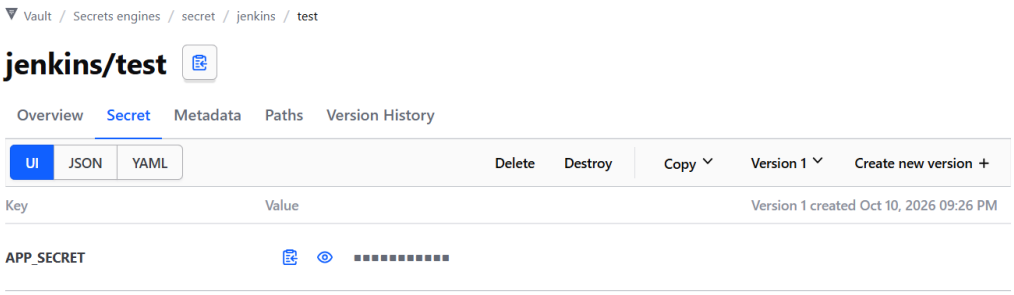


Una vez tengamos esto colocamos el secreto dentro de secret

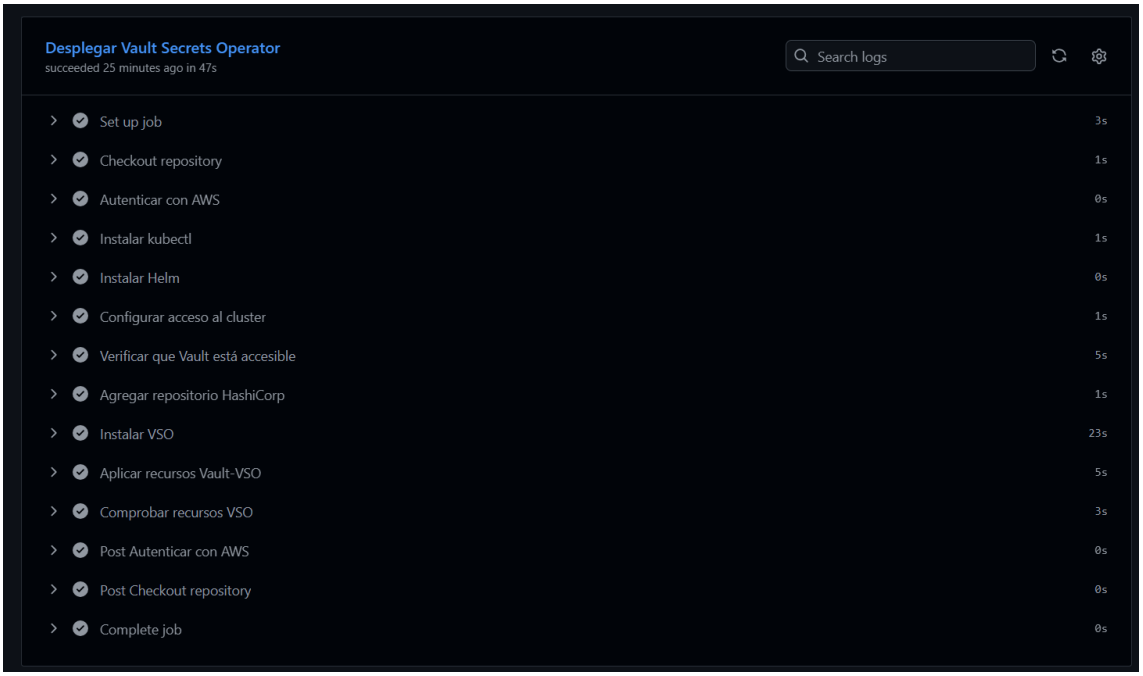


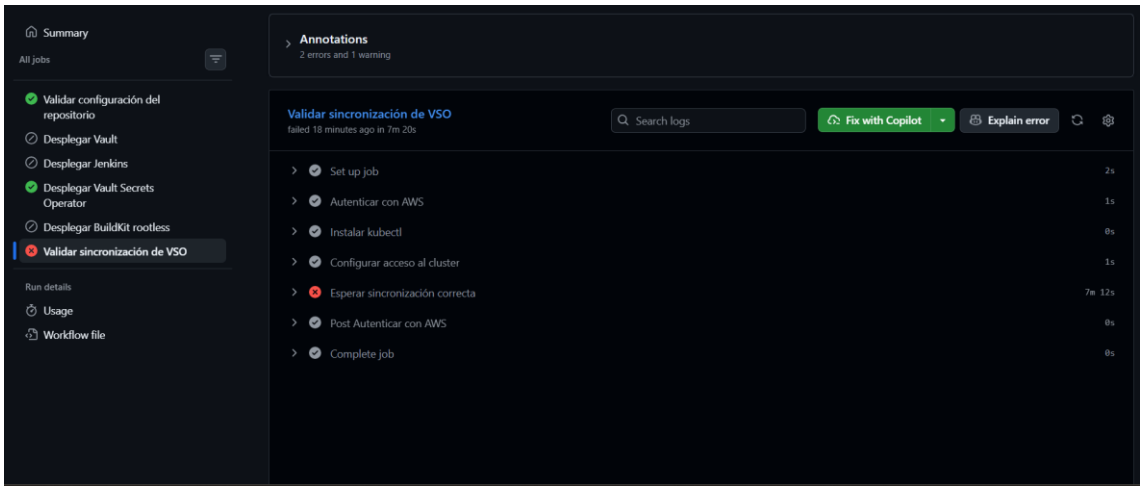




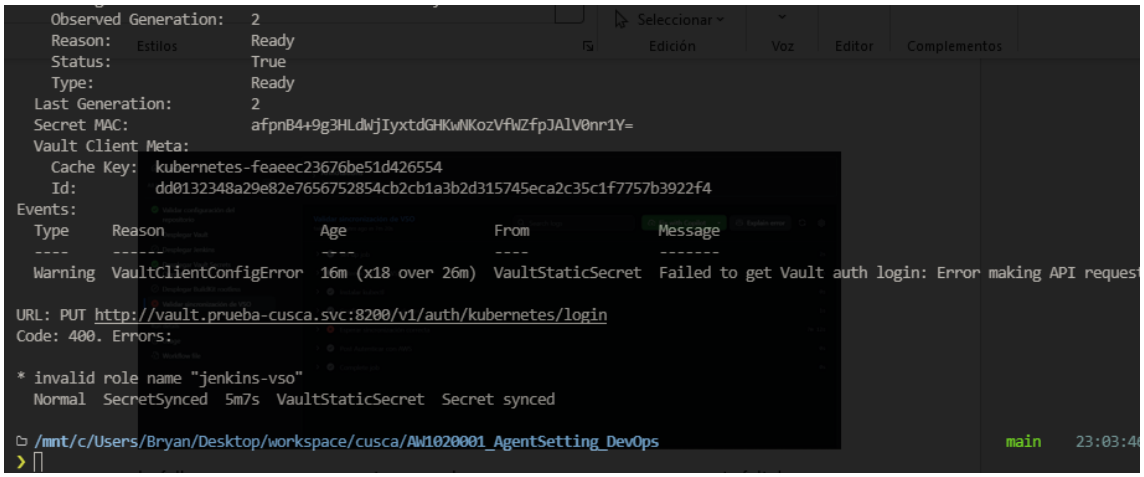


Esto como prueba de que se ha ido creando y operando sobre los recursos, se han creado otros roles y políticas bajo la necesidad para VSO que se ha implementado por medio de pipeline, a continuación, el proceso de deploy

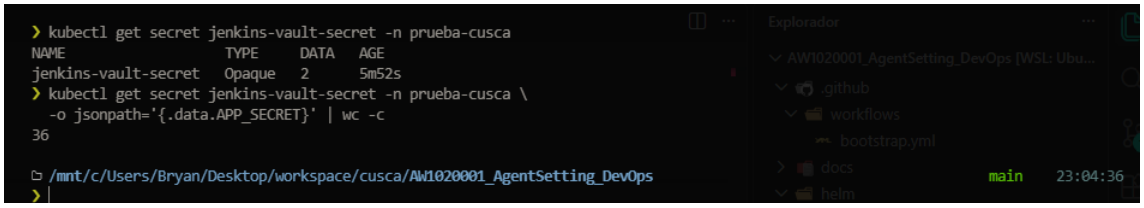




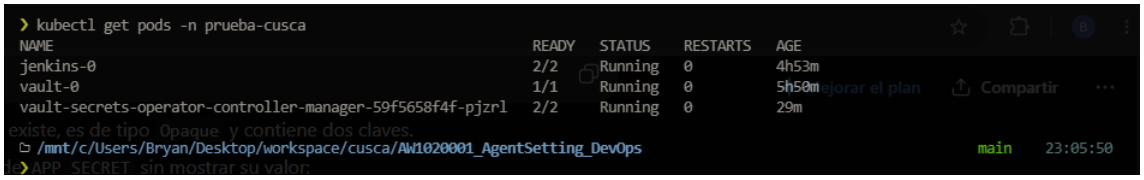
La falla que vemos es por una tarea que hace una espera, para ese momento faltaba un role no definido y se creó, se hizo una leve espera para que VSO refrescara en el cluster y se disponibilizara, se muestra a continuación



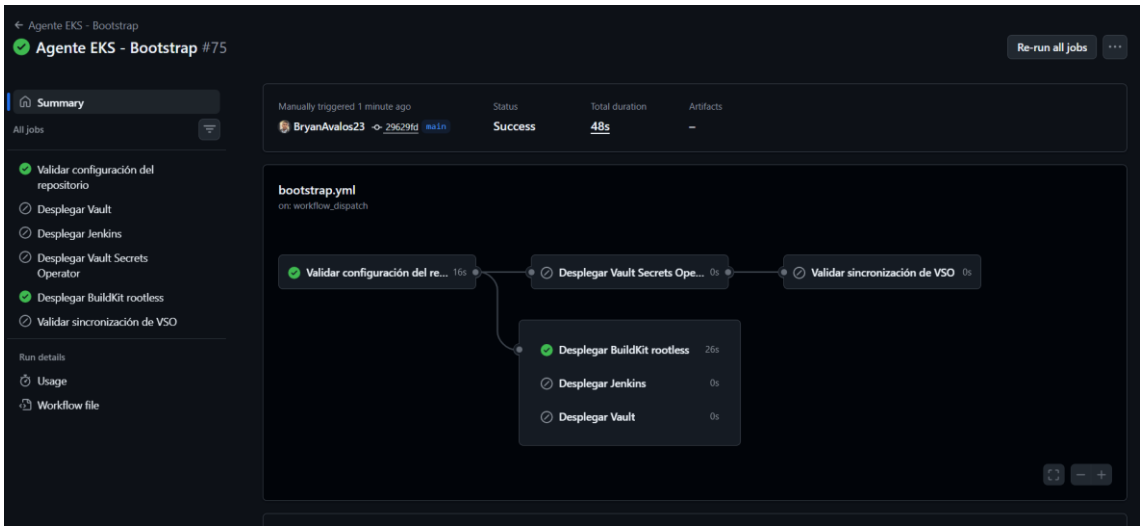
Validamos los siguientes comandos para verificar, donde vemos que por VSO ya obtiene los secretos

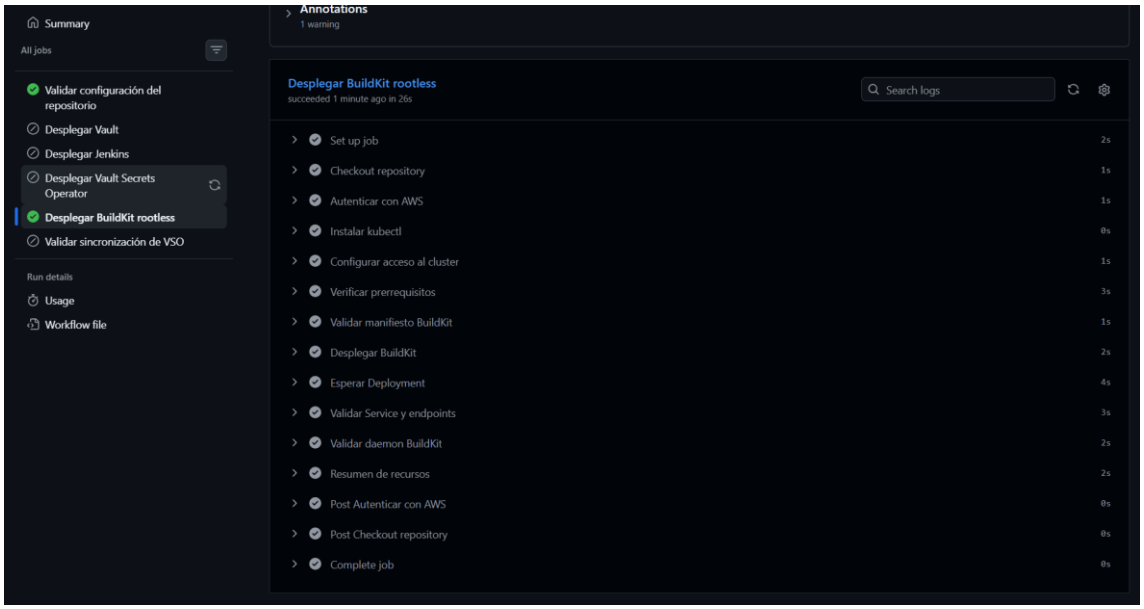


Validamos que este corriendo el pod



Hoy procedemos al deploy de buildkit





Validamos que los pods estén funcionales



Habilitamos port forward para el buildkit

```
❯ kubectl port-forward -n prueba-cusca svc/b
uildkit 1234:1234
Forwarding from 127.0.0.1:1234 -> 1234
Forwarding from [::1]:1234 -> 1234
```

Validamos que tenga worker

```
❯ kubectl exec -n prueba-cusca deployment/buildkit -- \
  buildctl --addr tcp://127.0.0.1:1234 debug workers
ID                          PLATFORMS
um0ssx5nzf8w68i86dzon3rhh   linux/amd64,linux/amd64/v2,linux/amd64/v3,linux/amd64/v4,linux/386

🗀 /mnt/c/Users/Bryan/Desktop/workspace/cusca/AW1020001_AgentSetting_DevOps
❯
```

Test ejecutando prueba

```
❯ kubectl run buildkit-client-test \
  -n prueba-cusca \
  --image=moby/buildkit:v0.25.1-rootless \
  --restart=Never \
  --command -- sh -c '
    set -eu

    mkdir -p /tmp/context
    printf "Hola desde BuildKit en EKS\n" > /tmp/context/message.txt

    cat > /tmp/context/Dockerfile <<EOF
FROM scratch
COPY message.txt /message.txt
EOF

    echo "=== Iniciando build OCI ==="

    buildctl --addr tcp://buildkit:1234 build \
      --frontend dockerfile.v0 \
      --local context=/tmp/context \
      --local dockerfile=/tmp/context \
      --output type=oci,dest=/tmp/buildkit-test.tar

    test -s /tmp/buildkit-test.tar
    echo "=== BUILD EXITOSO ==="
    ls -lh /tmp/buildkit-test.tar
  '
```

Validamos

```
❯ kubectl logs buildkit-client-test -n prueba-cusca
=== Iniciando build OCI ===
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 133B done
#1 DONE 0.0s

#2 [internal] load .dockerignore
#2 transferring context: 2B done
#2 DONE 0.0s

#3 [internal] load build context
#3 transferring context: 118B done
#3 DONE 0.0s

#4 [1/1] COPY message.txt /message.txt
#4 DONE 0.0s

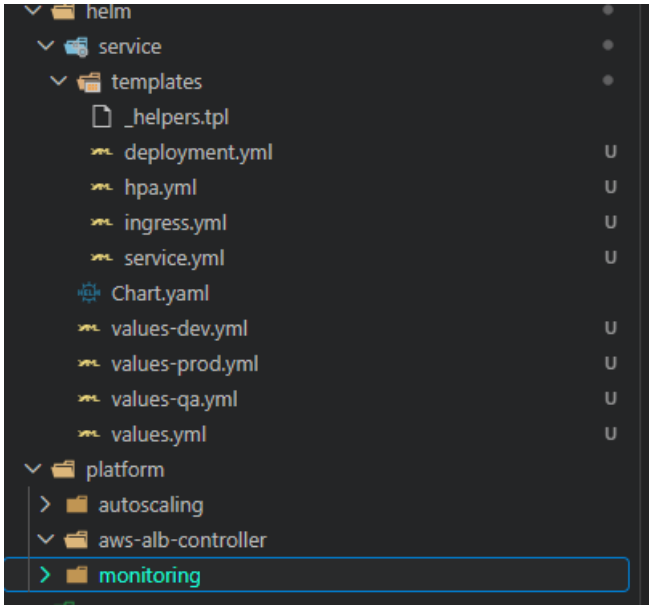
#5 exporting to oci image format
#5 exporting layers 0.1s done
#5 exporting manifest sha256:52481eb52a96d0c3fe63ab5e36bb9399af81d450fb8070879c2900a7e856ffc0 0.0s done
#5 exporting config sha256:6fb9f9ccdf5ddb4a03748ab94c0c20e8df1c43e7415e8c350e3024e167fe54f6 0.0s done
#5 sending tarball done
#5 DONE 0.1s
=== BUILD EXITOSO ===
-rw-rw-rw-    1 user     user       7.0K Oct 11 05:25 /tmp/buildkit-test.tar

🗀 /mnt/c/Users/Bryan/Desktop/workspace/cusca/AW1020001_AgentSetting_DevOps
❯
```

Como siguiente paso, para configurar el cluster de aplicativo, nos conectamos al Cluster y se dejó un namespace específico para este

```
❯ kubectl get namespaces
NAME                STATUS   AGE
default             Active   11h
demo-service-prod   Active   12m
kube-node-lease     Active   11h
kube-public         Active   11h
kube-system         Active   11h
```

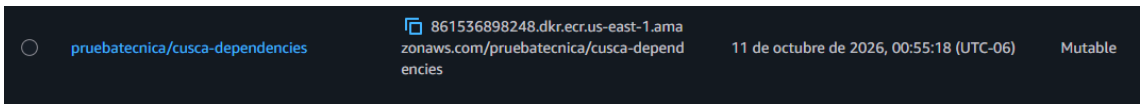
Luego de esto se configuraron los manifiestos en el repositorio de código del aplicativo



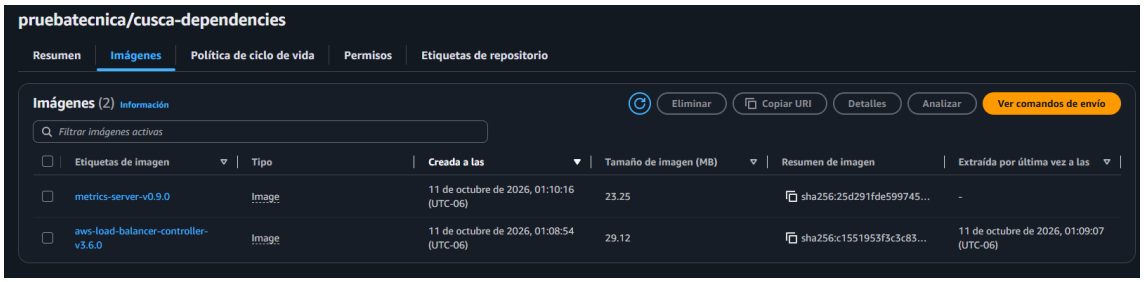
Seguido de esto, se descargaron las imágenes de load balancer controller y metrics server, esto debido que el cluster de aplicativo no tiene salida a internet para obtenerlos, por lo cual se agregaron a un ECR privado y por medio de VPC Endpoints se comunica de forma segura

```
IMAGE
861536898248.dkr.ecr.us-east-1.amazonaws.com/pruebatecnica/cusca-dependencies:aws-load-balancer-controller-v3.6.0
861536898248.dkr.ecr.us-east-1.amazonaws.com/pruebatecnica/cusca-dependencies:metrics-server-v0.9.0
```

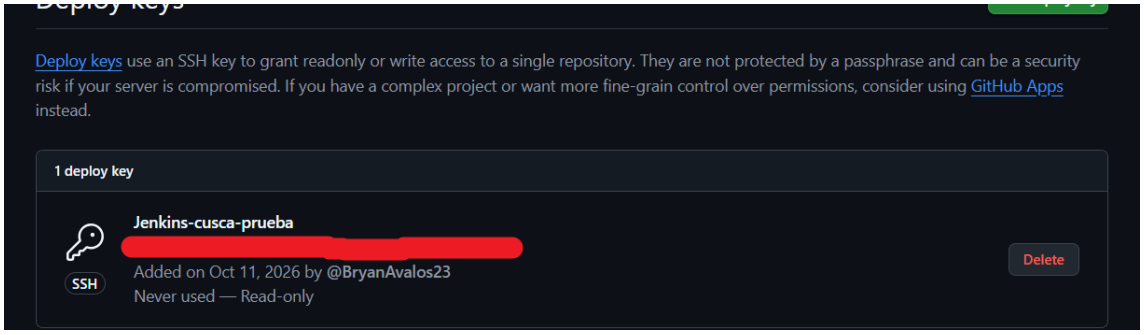
Se creo un nuevo ECR específico para este propósito



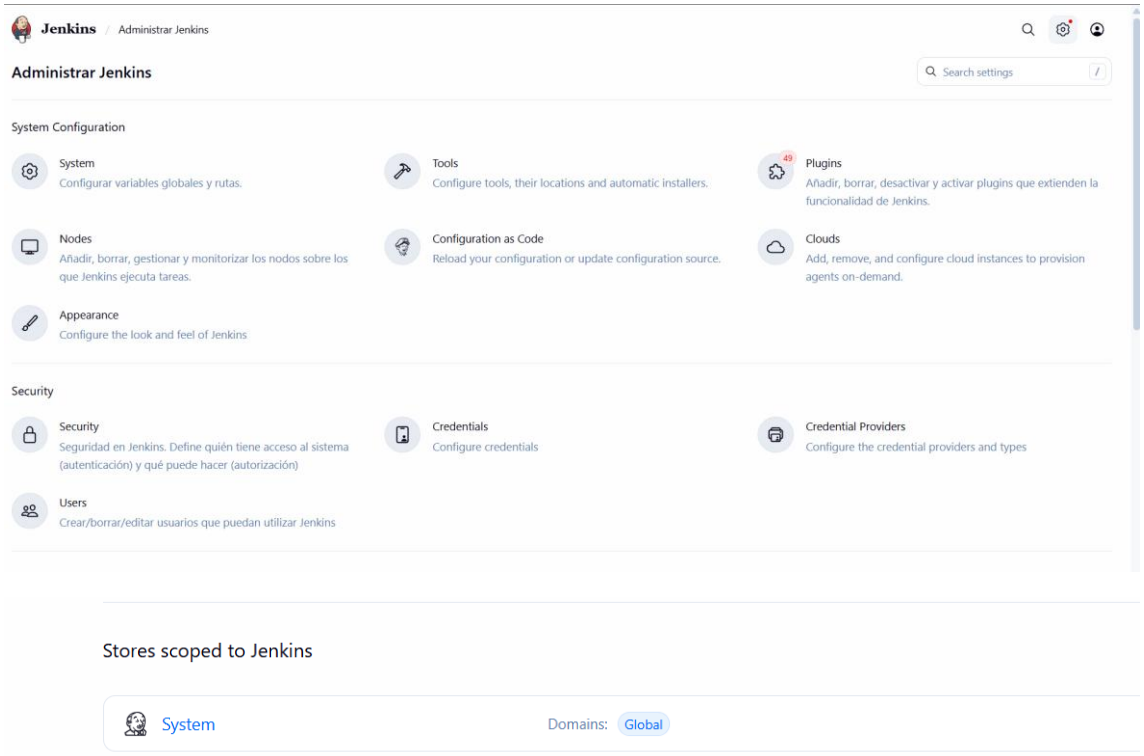
Validamos la existencia de las imágenes necesarias



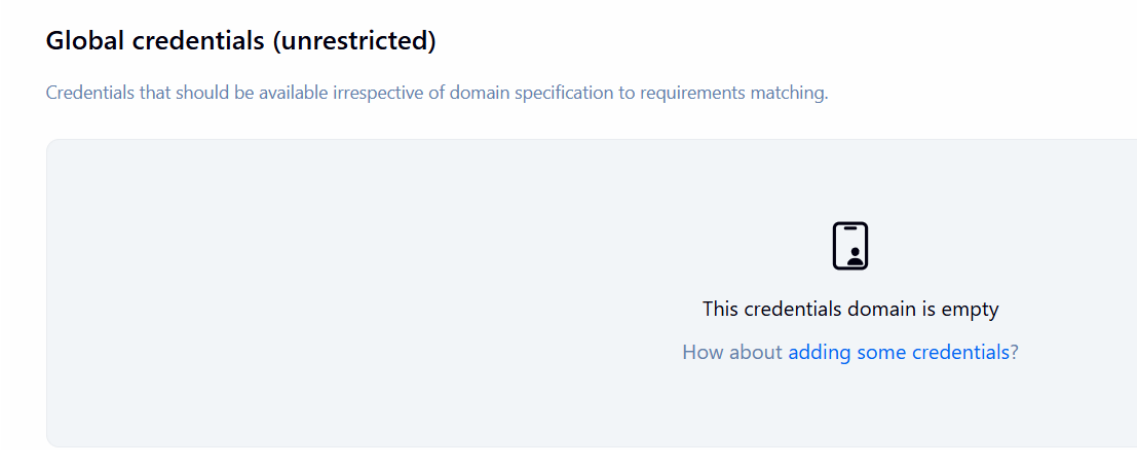
Ahora configuramos el repositorio de código en Jenkins, actualmente el código está ubicado en GitHub, para esto primero creamos una llave ssh a utilizar en Jenkins y Github

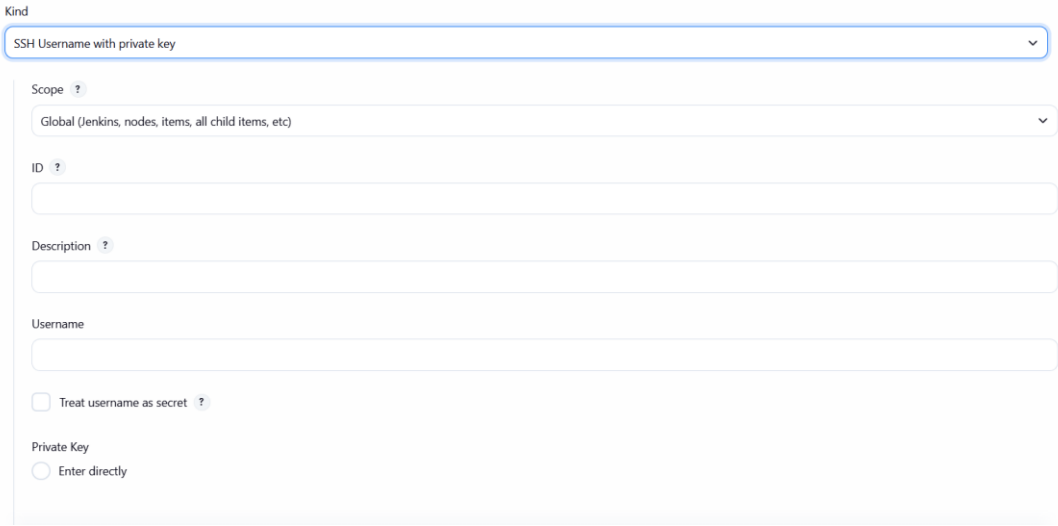


Luego de esto configuramos las credenciales en Jenkins con la llave privada, en settings > credentials > global

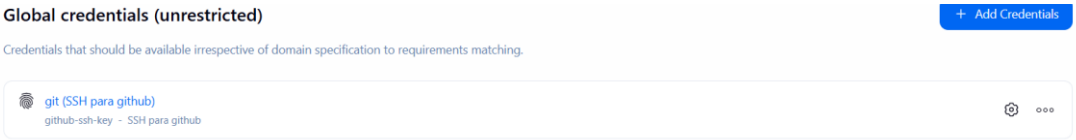


Seleccionamos adding some credentials y seleccionaremos la opción por ssh, colocamos los datos necesarios

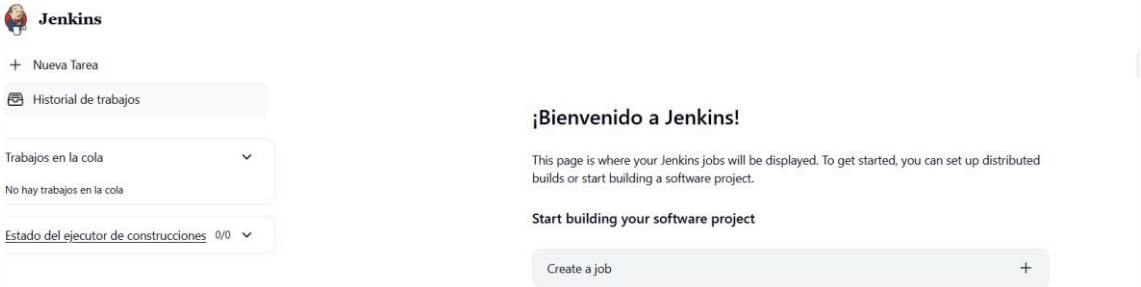




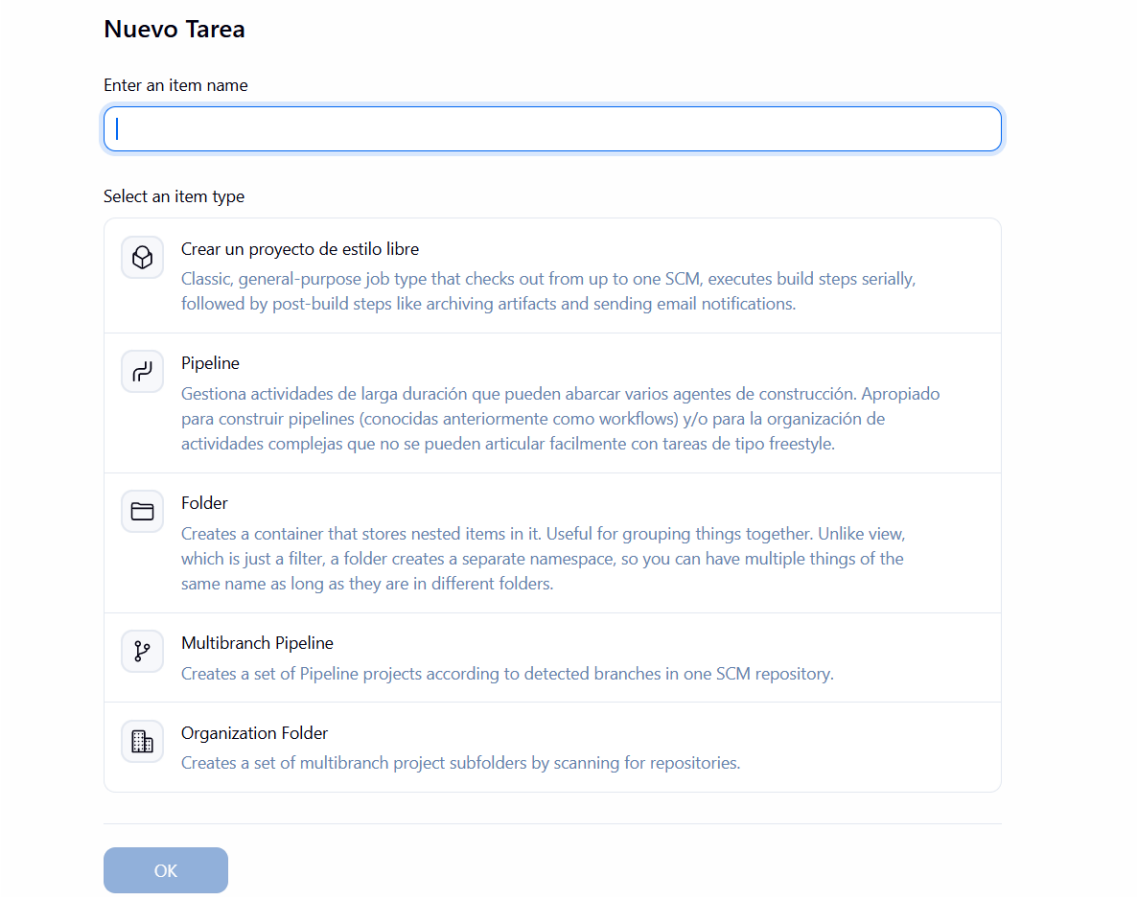
Una vez creada veremos esto

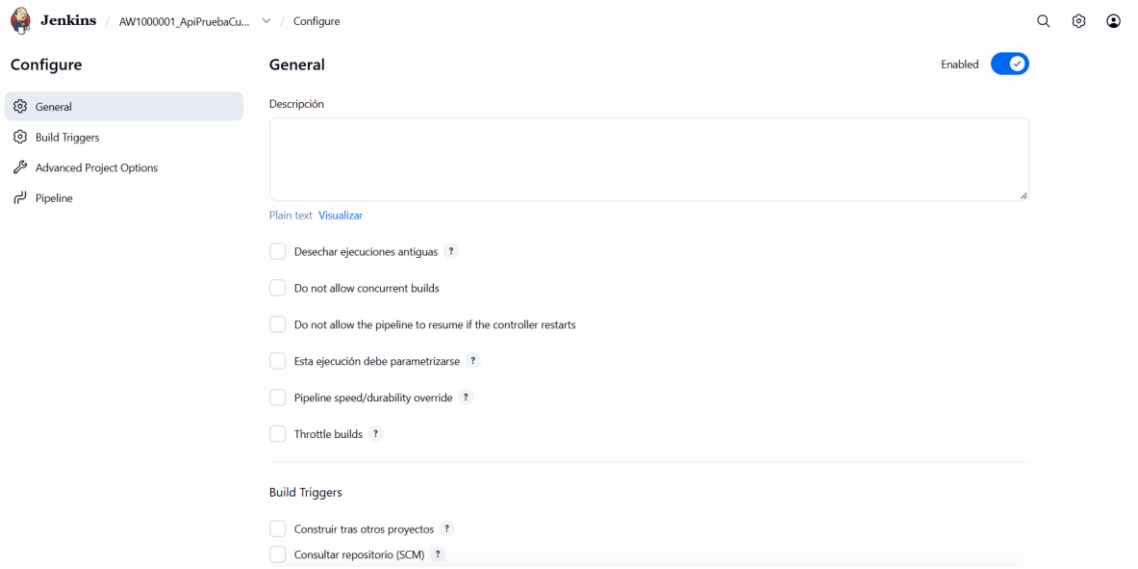


Ahora configuramos el pipeline y el repo donde está ubicado, cambiaremos a el home y seleccionaremos créate job o Nueva tarea

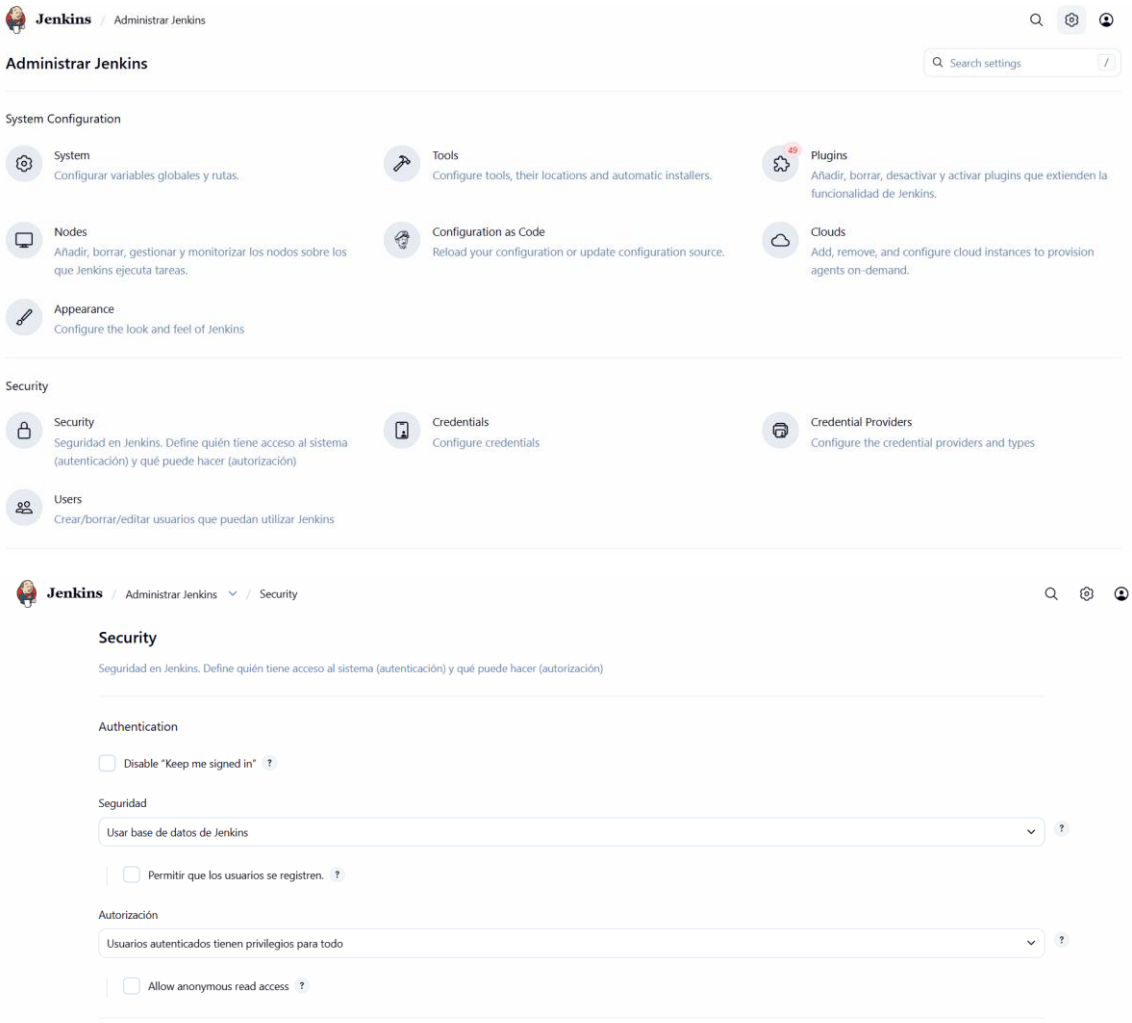


Colocamos el nombre y seleccionamos pipeline, una vez completemos eso damos OK y veremos el pipeline creado

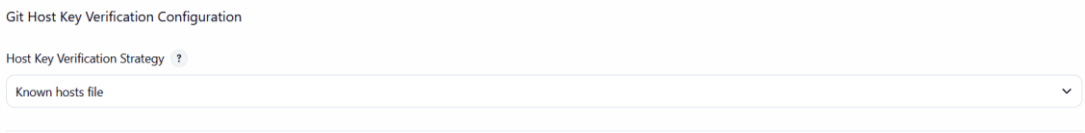




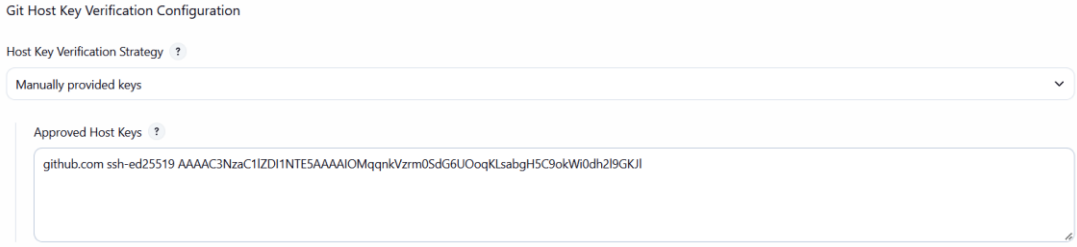
Luego nos vamos a manage nuevamente para configurar el key de verificación



Aquí buscaremos el siguiente apartado



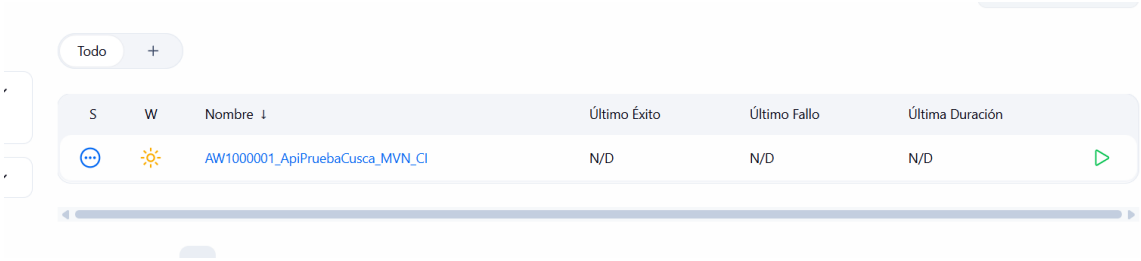
Colocaremos lo siguiente



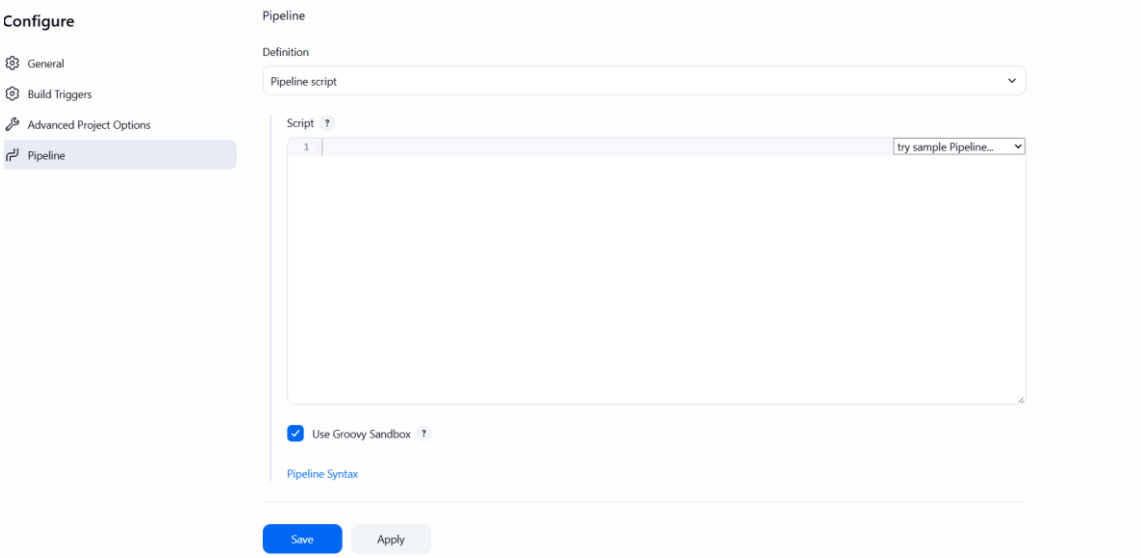
Para saber que host key colocar visita este sitio

https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/githubs-ssh-key-fingerprints

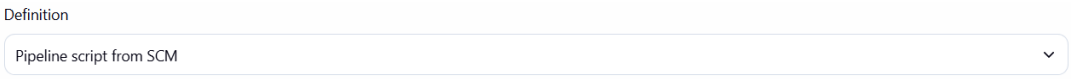
Hoy configuramos lo necesario para consumir el archivo desde GitHub, buscamos el apartado pipeline, regresamos al home y veremos el pipeline



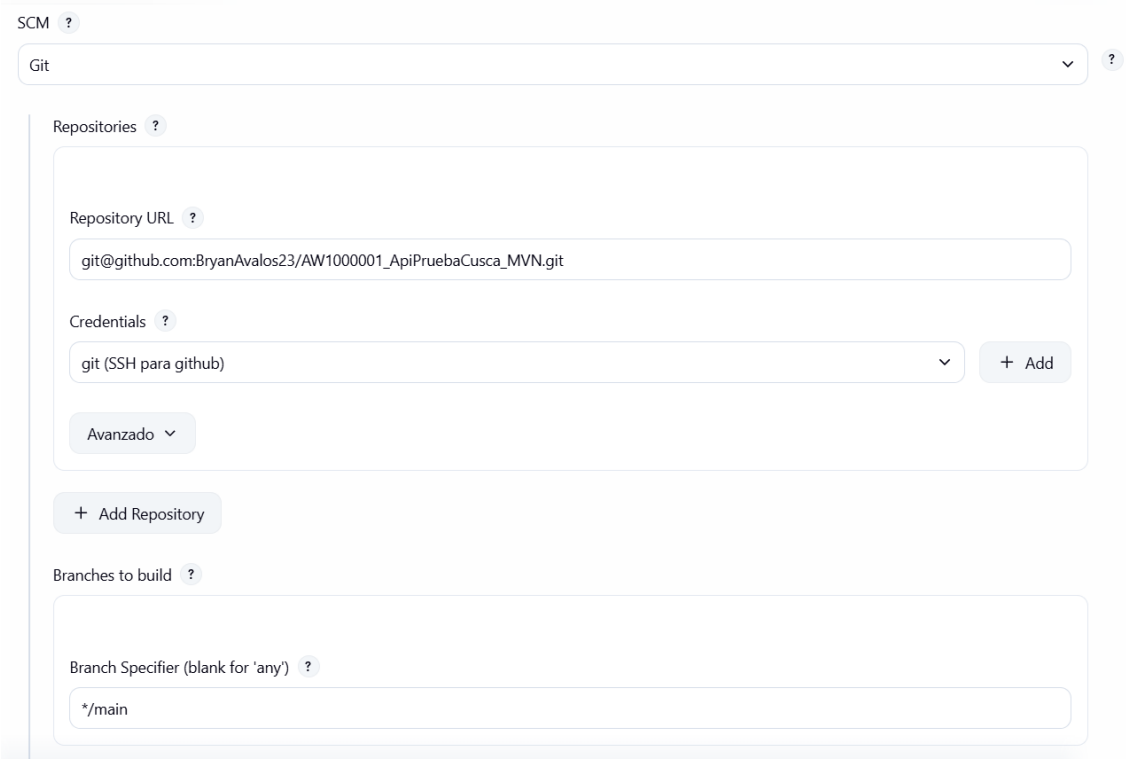
Luego vamos al siguiente apartado configure > pipeline



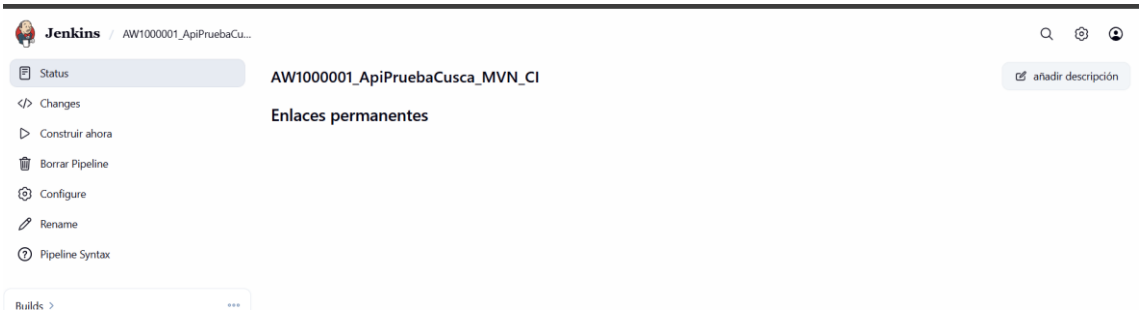
Luego cambiamos la opción por



Luego configuramos el acceso al repositorio, coloca los datos según tu repositorio, damos en save



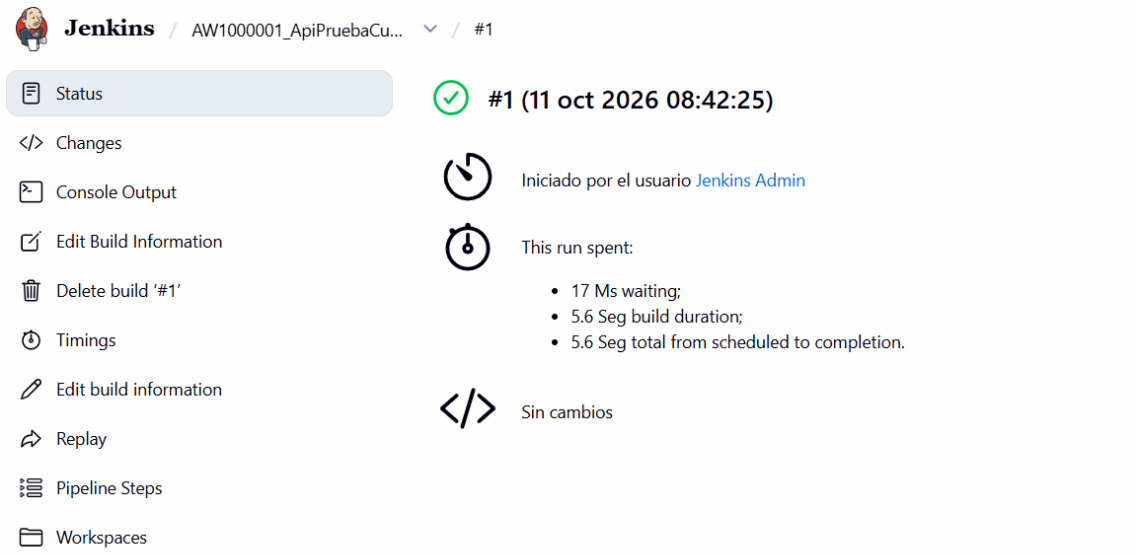
De momento no hay nada, por lo cual podemos intentar ejecutar, seleccionamos construir ahora



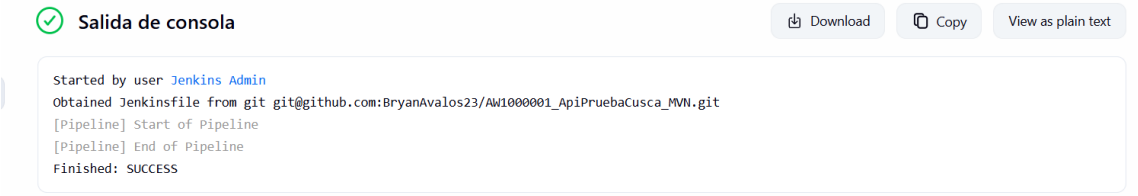
Para ver la ejecución seleccionamos el siguiente apartado



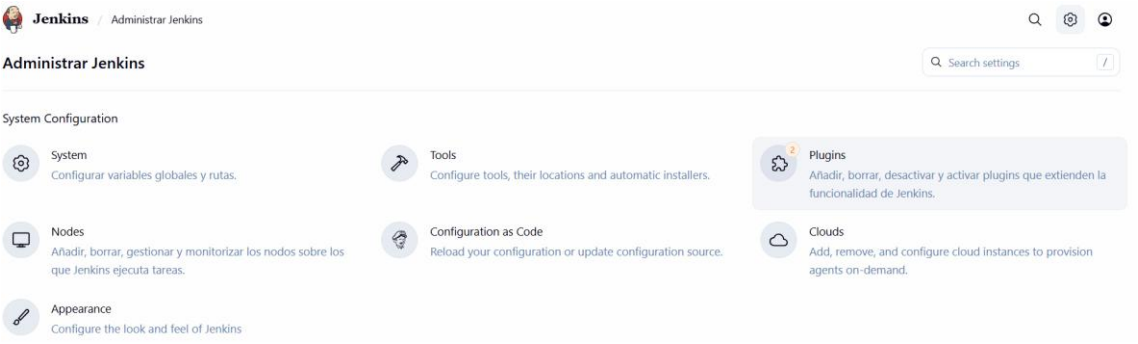
Veremos lo siguiente

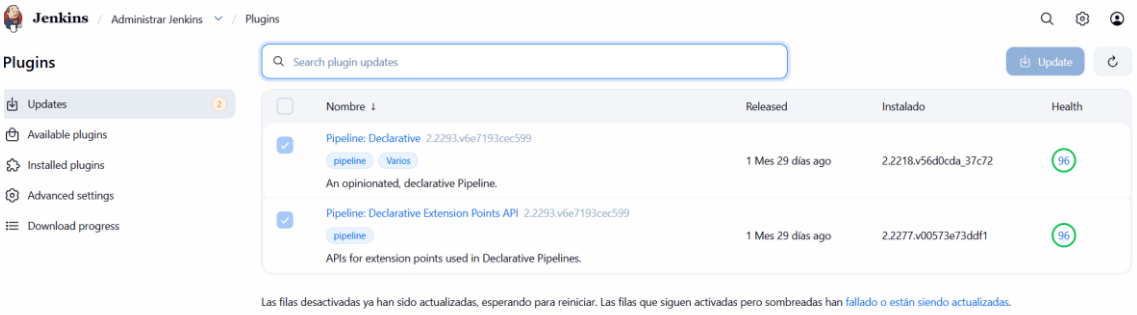


Para ver el paso a paso nos vamos a console output



Si vemos que se queda sin avanzar el pipeline hay que validar los pluggins ya que muchas veces estos no permiten hacer varias cosas por estar desactualizados, para eso en settings > plugins





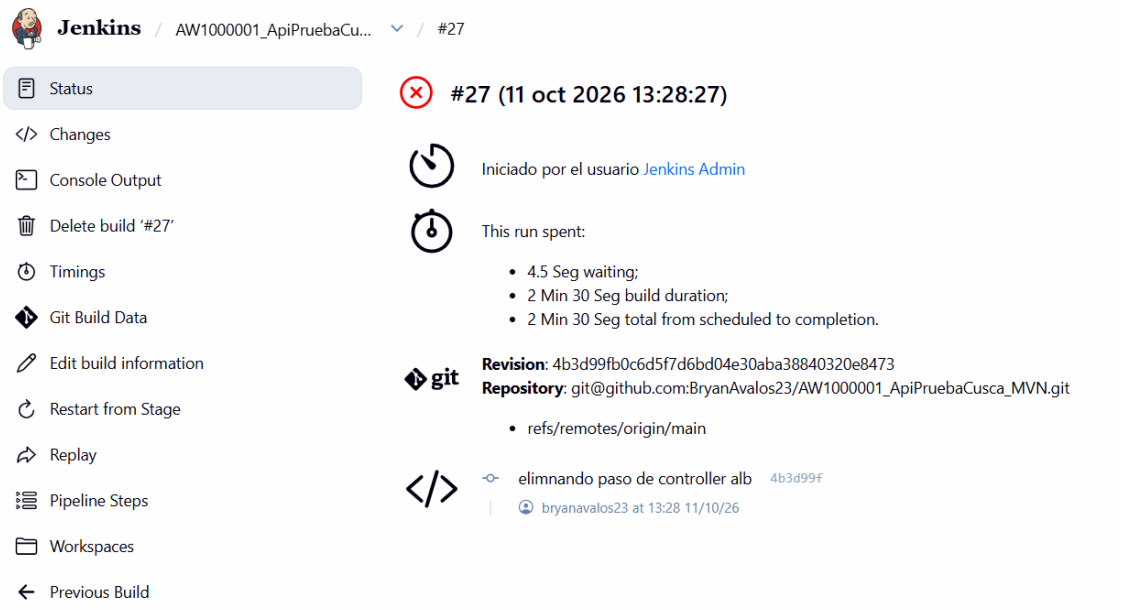
Una vez este instaladas las actualizaciones Jenkins pide reiniciar el servicio, una vez listo eso proseguimos con el pipeline, en este caso se completo el deploy presentando varios problemas y depurando algunos stage del pipeline

```
Started by user Jenkins Admin
Obtained Jenkinsfile from git git@github.com:BryanAvalos23/AW1000001_ApiPruebaCusca_MVN.git
[Pipeline] Start of Pipeline (hide)
[Pipeline] podTemplate
[Pipeline] {
[Pipeline] node
Created Pod: kubernetes prueba-cusca/aw1000001-apipruebacusca-mvn-27-hp463-7kj43-k0zcb
[PodInfo] prueba-cusca/aw1000001-apipruebacusca-mvn-27-hp463-7kj43-k0zcb
        Container [aws] waiting [ContainerCreating] No message
        Container [buildkit] waiting [ContainerCreating] No message
        Container [helm] waiting [ContainerCreating] No message
        Container [jnlp] waiting [ContainerCreating] No message
        Container [maven] waiting [ContainerCreating] No message
        Pod [Pending][ContainersNotReady] containers with unready status: [maven aws buildkit helm jnlp]
[PodInfo] prueba-cusca/aw1000001-apipruebacusca-mvn-27-hp463-7kj43-k0zcb
        Container [aws] waiting [ContainerCreating] No message
        Container [buildkit] waiting [ContainerCreating] No message
        Container [helm] waiting [ContainerCreating] No message
        Container [jnlp] waiting [ContainerCreating] No message
        Container [maven] waiting [ContainerCreating] No message
        Pod [Pending][ContainersNotReady] containers with unready status: [maven aws buildkit helm jnlp]
[PodInfo] prueba-cusca/aw1000001-apipruebacusca-mvn-27-hp463-7kj43-k0zcb
        Container [aws] waiting [ContainerCreating] No message
        Container [buildkit] waiting [ContainerCreating] No message
        Container [helm] waiting [ContainerCreating] No message
        Container [jnlp] waiting [ContainerCreating] No message
        Container [maven] waiting [ContainerCreating] No message
        Pod [Pending][ContainersNotReady] containers with unready status: [maven aws buildkit helm jnlp]
[PodInfo] prueba-cusca/aw1000001-apipruebacusca-mvn-27-hp463-7kj43-k0zcb
```

```
REVISION: 1
TEST SUITE: None
=== Application release status ===
NAME: demo-service
LAST DEPLOYED: Sun Oct 11 13:30:19 2026
NAMESPACE: demo-service-prod
STATUS: deployed
REVISION: 1
TEST SUITE: None
Application deployment with Ingress completed.
[Pipeline] }
[Pipeline] // container
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finalized. Kubernetes will remove the ephemeral agent pod.
[Pipeline] echo
Pipeline completed: tests passed, image published, controller installed, and application deployed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // container
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] }
[Pipeline] // podTemplate
```

En el pipe, sale un error al final, pero es relacionado con grovy, nada que sea dependencia de nuestra app ya que es con Maven

```
java.lang.NullPointerException: Cannot invoke method call() on null object
        at org.codehaus.groovy.runtime.NullObject.invokeMethod(NullObject.java:91)
        at org.codehaus.groovy.runtime.callsite.PogoMetaClassSite.call(PogoMetaClassSite.java:47)
        at org.codehaus.groovy.runtime.callsite.CallSiteArray.defaultCall(CallSiteArray.java:47)
        at org.codehaus.groovy.runtime.callsite.NullCallSite.call(NullCallSite.java:34)
        at org.codehaus.groovy.runtime.callsite.CallSiteArray.defaultCall(CallSiteArray.java:47)
        at org.codehaus.groovy.runtime.callsite.AbstractCallSite.call(AbstractCallSite.java:116)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.sandbox.DefaultInvoker.methodCall(DefaultInvoker.java:20)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.LoggingInvoker.methodCall(LoggingInvoker.java:124)
        at org.jenkinsci.plugins.pipeline.modeldefinition.ModelInterpreter.inDeclarativeAgent(ModelInterpreter.groovy:601)
        at org.jenkinsci.plugins.pipeline.modeldefinition.ModelInterpreter.call(ModelInterpreter.groovy:77)
        at WorkflowScript.run(WorkflowScript:1)
        at ___cps.transform___(Native Method)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.impl.ContinuationGroup.methodCall(ContinuationGroup.java:107)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.impl.FunctionCallBlock$ContinuationImpl.dispatchOrArg(FunctionCallBlock.java:118)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.impl.FunctionCallBlock$ContinuationImpl.fixName(FunctionCallBlock.java:82)
        at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(Unknown Source)
        at java.base/java.lang.reflect.Method.invoke(Unknown Source)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.impl.ContinuationPtr$ContinuationImpl.receive(ContinuationPtr.java:71)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.impl.ConstantBlock.eval(ConstantBlock.java:21)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.Next.step(Next.java:84)
        at PluginClassLoader for workflow-cps//com.cloudbees.groovy.cps.Continuable.run0(Continuable.java:142)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.SandboxContinuable.access$001(SandboxContinuable.java:17)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.SandboxContinuable.run0(SandboxContinuable.java:48)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsThread.runNextChunk(CpsThread.java:188)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsThreadGroup.run(CpsThreadGroup.java:464)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsThreadGroup$2.call(CpsThreadGroup.java:372)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsThreadGroup$2.call(CpsThreadGroup.java:302)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsVmExecutorService.lambda$wrap$4(CpsVmExecutorService.java:143)
        at java.base/java.util.concurrent.FutureTask.run(Unknown Source)
        at hudson.remoting.SingleLaneExecutorService$1.run(SingleLaneExecutorService.java:139)
        at jenkins.util.ContextResettingExecutorService.lambda$wrap$0(ContextResettingExecutorService.java:26)
        at jenkins.security.ImpersonatingExecutorService.lambda$wrap$0(ImpersonatingExecutorService.java:66)
        at jenkins.util.ErrorLoggingExecutorService.lambda$wrap$0(ErrorLoggingExecutorService.java:51)
        at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Unknown Source)
        at java.base/java.util.concurrent.FutureTask.run(Unknown Source)
        at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(Unknown Source)
        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(Unknown Source)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsVmExecutorService$1.call(CpsVmExecutorService.java:53)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsVmExecutorService$1.call(CpsVmExecutorService.java:50)
        at org.codehaus.groovy.runtime.GroovyCategorySupport$ThreadCategoryInfo.use(GroovyCategorySupport.java:136)
        at org.codehaus.groovy.runtime.GroovyCategorySupport.use(GroovyCategorySupport.java:275)
        at PluginClassLoader for workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsVmExecutorService.lambda$categoryThreadFactory$0(CpsVmExecutorService.java:50)
        at java.base/java.lang.Thread.run(Unknown Source)
Finished: FAILURE
```



Aun así, no hace afectación ya que el deploy se completo como se ve en las imágenes anteriores

Una vez desplegado, listaremos los pods y tendríamos que ver esto

```
❯ kubectl get nodes -o wide
NAME                          STATUS   ROLES    AGE   VERSION              INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                         KERNE
L-VERSION                     CONTAINER-RUNTIME
ip-10-0-20-147.ec2.internal   Ready    <none>   21h   v1.36.4-eks-3b4a6ca   10.0.20.147   <none>        Amazon Linux 2023.12.20260930   6.18.
51-120.163.amzn2023.x86_64 (amd64)   containerd://2.2.7+unknown
ip-10-0-21-133.ec2.internal   Ready    <none>   13h   v1.36.4-eks-3b4a6ca   10.0.21.133   <none>        Amazon Linux 2023.12.20260930   6.18.
51-120.163.amzn2023.x86_64 (amd64)   containerd://2.2.7+unknown
```

Listamos deployment, pods y servicios

```
❯ kubectl get deployment,pods,svc -n demo-service-prod -o wide
NAME                           READY   UP-TO-DATE   AVAILABLE   AGE     CONTAINERS     IMAGES
                  SELECTOR
deployment.apps/demo-service   2/2     2            2           3h52m   demo-service   861536898248.dkr.ecr.us-east-1.amazonaws.com/pruebatec
nica/cusca:latest   app.kubernetes.io/instance=demo-service,app.kubernetes.io/name=demo-service

NAME                                READY   STATUS    RESTARTS   AGE     IP            NODE                           NOMINATED NODE   READINE
SS GATES
pod/demo-service-5445fc9467-j4bzp   1/1     Running   0          3h52m   10.0.21.197   ip-10-0-21-133.ec2.internal   <none>           <none>
pod/demo-service-5445fc9467-xrk9b   1/1     Running   0          3h52m   10.0.20.39    ip-10-0-20-147.ec2.internal   <none>           <none>

NAME                   TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE     SELECTOR
service/demo-service   ClusterIP   172.20.130.184   <none>        8081/TCP   3h52m   app.kubernetes.io/instance=demo-service,app.kubernetes.i
o/name=demo-service
```

La configuración del ALB controller se hace manual desde consola aplicando los manifiestos, esto debido que generaba percances al momento de automatizar y reiniciaba algunas configuraciones

```
❯ kubectl get deployment,pods -n kube-system \
  -l app.kubernetes.io/name=aws-load-balancer-controller -o wide
NAME                                           READY   UP-TO-DATE   AVAILABLE   AGE     CONTAINERS                    IMAGES
                                                                                                SELECTOR
deployment.apps/aws-load-balancer-controller   2/2     2            2           4h58m   aws-load-balancer-controller   861536898248.dkr.ecr.u
s-east-1.amazonaws.com/pruebatecnica/cusca-dependencies:aws-load-balancer-controller-v3.6.0   app.kubernetes.io/instance=aws-load-balancer-co
ntroller,app.kubernetes.io/name=aws-load-balancer-controller

NAME                                            READY   STATUS    RESTARTS   AGE     IP            NODE                           NOMINATED
 NODE   READINESS GATES
pod/aws-load-balancer-controller-7fc5957f5-8jfkg   1/1     Running   0          4h58m   10.0.20.134   ip-10-0-20-147.ec2.internal   <none>
        <none>
pod/aws-load-balancer-controller-7fc5957f5-kl8wj   1/1     Running   0          4h58m   10.0.21.225   ip-10-0-21-133.ec2.internal   <none>
        <none>

/mnt/c/Users/Bryan/Desktop/workspace/cusca/Prueba tecnica                                                                    11:24:25
❯
```

Validamos el ingress con un ALB de acceso publico

```
❯ kubectl get ingress -n demo-service-prod -o wide
NAME           CLASS   HOSTS   ADDRESS                                                                  PORTS   AGE
demo-service   alb     *       k8s-demoserv-demoserv-069eb8ba89-304355471.us-east-1.elb.amazonaws.com   80      3h54m

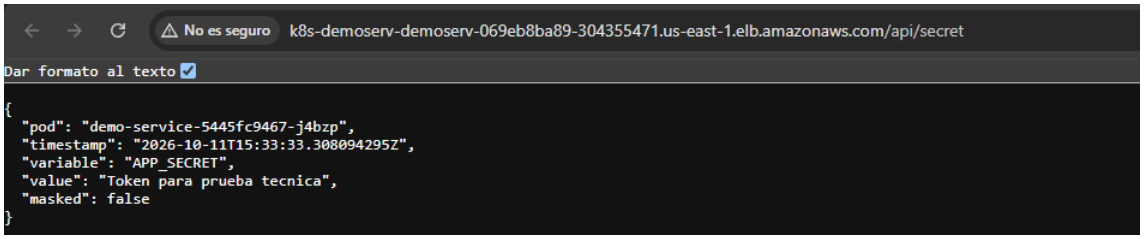
🗀 /mnt/c/Users/Bryan/Desktop/workspace/cusca/Prueba tecnica
❯
```

Hacemos validación de funcionamiento al alb

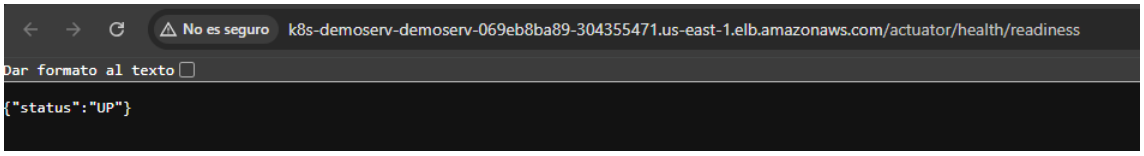
```
❯ curl -i http://k8s-demoserv-demoserv-069eb8ba89-304355471.us-east-1.elb.amazonaws.com/actuator/health/readiness
HTTP/1.1 200
Date: Sun, 11 Oct 2026 17:25:51 GMT
Content-Type: application/vnd.spring-boot.actuator.v3+json
Transfer-Encoding: chunked
Connection: keep-alive

{"status":"UP"}
```

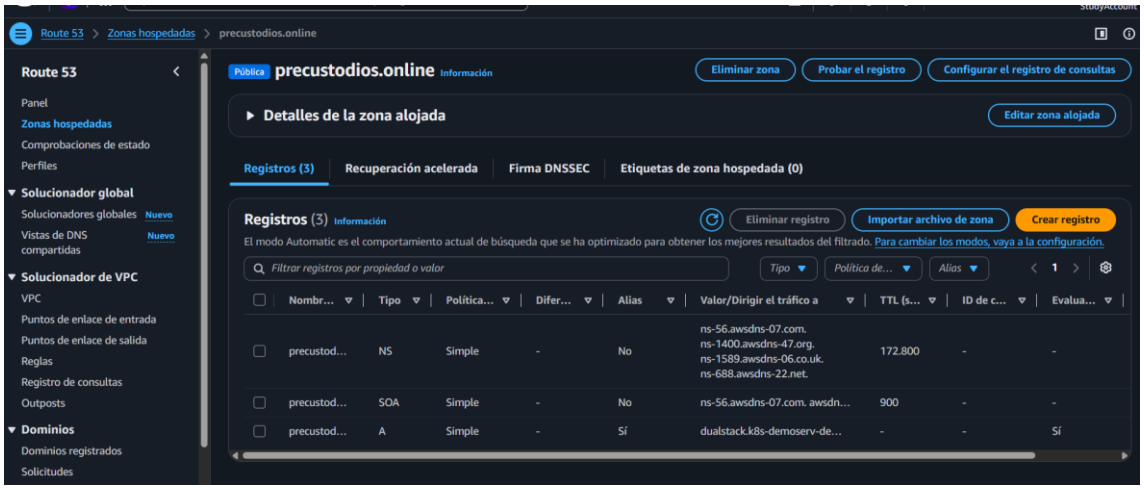
Y desde el navegador



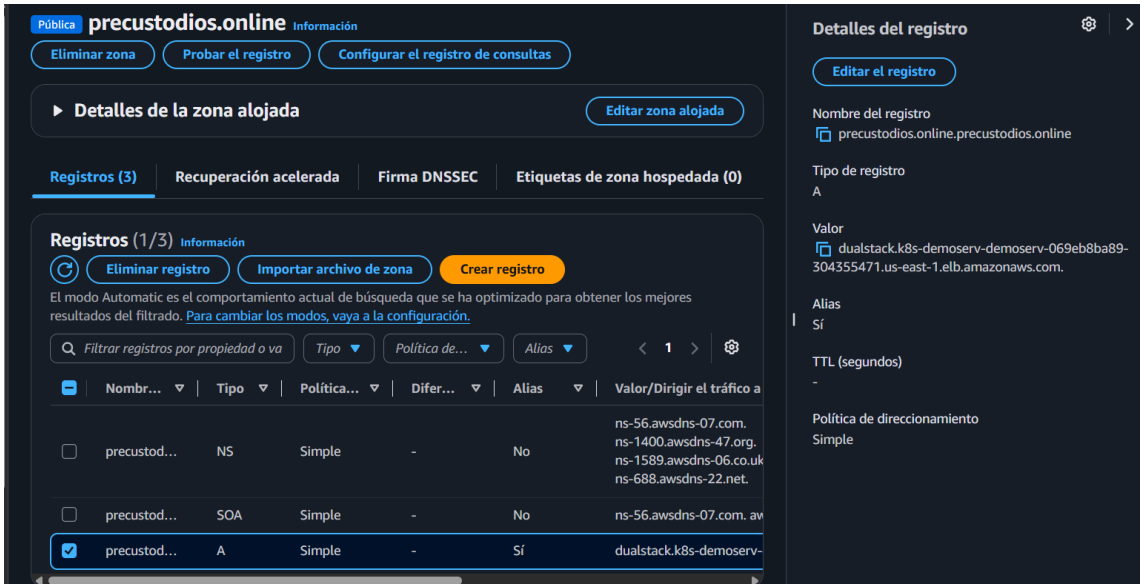




Ahora para darle un toque mas amigable configuramos un dominio en Route 53 y aplicamos un certificado de ACM para seguridad TLS, recuerda cambiar los servidores de nombre en tu proveedor de dominios



Luego agregamos un registro de tipo A que apunta hacia nuestro ALB



Creamos un certificado en ACM este pedirá el dominio, y luego de esto esperamos la confirmación DNS en Route53 para que quede validado y tengamos https



Una vez este listo validamos usando el dominio que acabamos de asignar